THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

THE UNIVERSITY OF QUEENSLAND

Mathematical and Computational Modelling of the Self-Organisation of Multicellular Tissues

Student Name: Madeleine, FRASER

Course Code: ENGG4600

Supervisor: Dr Hamid Khataee

Submission date: 28 October 2021

Faculty of Engineering, Architecture and Information Technology

THE UNIVERSITY OF QUEENSLAND

AUSTRALIA

Mathematical and Computational Modelling of the Self-Organisation of Multicellular Tissues

Madeleine Fraser

Supervisor: Dr Hamid Khataee

October, 2021

Abstract

The behaviour of collective cell formation, migration and proliferation underpins tissue morphology. Epithelial sheets in particular play a vital role in healthy tissue development and cancer progression. This project aims to present a computational model for analysing the development of collective epithelial morphology. The model is developed based on the Cellular Potts Model and the computational simulations are implemented in the software package CompuCell3D.

The computational model presented in this thesis maps out phase diagrams as functions of cellular properties and the orientation of cell division. Monolayers of flat, cuboidal and tall cells were found when the axis of cell proliferation is perpendicular to the substrate. The monolayer-to-multilayer transition was found to be promoted via cell extrusion, depending on the mechanical properties of cells and the orientation of cell division. These results were considered against experimental observations and found to be consistent with past studies and biological cell experiments. These results characterise cellular properties and mechanics that affect tissue geometry, and thus can provide new insights for drug development and targeted therapies for disease associated with morphologies of epithelial tissues.

There is significant potential for further research building on this project in order to develop a deeper understanding of collective cell migration and the biophysical mechanisms under which cells feel their microenvironment and react through coordinated self-regulation.

Contents

1 Introduction

The coordination of collective cell movements underpins organ tissue development. Recent research has proven significant patterns between the behaviour of collective cell formation, migration and proliferation and their internal mechanisms [1], cellular interactions [2, 3] and geometrical constraints [4] in multicellular tissues within the cell microenvironment. Biological processes such as morphogenesis, the processes under which organ tissues develop their shape, is self-generated and based on the regulated coordination of cell populations.

While there has been recent experimental work in this area [5], the current understanding of these mechanisms and the conditions under which they operate are still largely unknown [6]. Core cellular properties, such as cell-cell adhesion, cell-substrate adhesion, cell cortex contractility and cell size have been shown to be self-regulating independently in cells and have significant effects on proliferation [7]. It has been theorised that the analysis and manipulation of these properties in combination could allow accurate modelling of cells through the formation of a variety of experimentally observed morphological structures. Previous projects have investigated similar areas of interest such as cell crowding and mutation using various cell types such as mutated [8] and multiscale regimes [9], but have yet to form these models from the proliferation of a single cell type on a substrate.

Cell monolayers are of significant interest in relation to tissue repair in cases of wound healing, cancerous cell invasion and organ function [10]. This project will examine the simplest tissue layer, the epithelium. This will be achieved through mathematical modelling of cellular dynamics represented by the Cellular Potts Model (CPM), based on the energy forms of the relevant cellular properties. Subsequent analysis through computational simulations of cell proliferation will use the modelling software CompuCell3D (CC3D). Computational simulations based on biophysical mathematical modelling is the ideal tool to support biological observations and ex-

perimental studies and provide new experimental directions. Numerical analysis based on these simulations will form the basis of our project to provide accurate predictions on cell proliferation and tissue morphology.

There is significant potential for this project to provide a new understanding of the relationship between the shapes of tissues and cellular mechanics, as this understanding can be applied to analyse biological processes such as cancer growth and organ tissue repair from internal wounds.

1.1 Research Questions

A general model that relates collective cell migration and and the mechanisms under which cells understand their microenvironment and react through coordinated self-regulation remains elusive. Therefore, the research focus for this project is to investigate the link between the three-dimensional (3D) structures of tissues and cellular dynamics. This will be achieved by two primary research questions.

1. How do the mechanical properties of cellular interactions determine cell aspect ratios and cell behaviour (in particular, the formation of mono- and multilayered epithelial structures)?

2. What is the role of the orientation of the plane of cell division in combination with different regulatory mechanisms in collective tissue morphology?

The project has been broken down into two research phases based on these two questions. The first task is to simulate the growth of a single cell on a substrate, limiting cell division, so as to explore the effects of internal cellular mechanisms on cell shape and size. The second phase of the project will then expand this analysis to multicellular formations, with particular focus on monolayered epithelial structures, taking into account the orientation of division as a major factor. From these, the

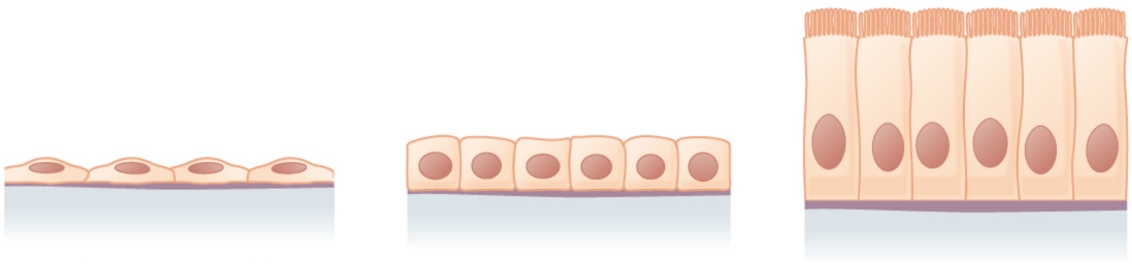
project acquires its main aim: to develop a simple computational model for analysing the development of collective epithelial morphology.

2 Literature Review

This section introduces the tissue structures that form the basis of our analysis, the epithelium. Following this, we introduce the internal cell mechanisms and intracellular dynamics that have been shown to govern collective cell behaviour. These factors will provide the framework for our computational modelling.

2.1 Epithelial Cells

Epithelial cell layers are the simplest living tissues that line organs throughout the body [11], yet contribute to 90% of all cancers [12]. Single layer epithelial cells, otherwise known as a simple epithelium, are the ideal ideal model for our project as they consist of simple structures and formations. These monolayer structures have been observed in flat (squamous), cuboidal and tall (columnar) formations, shown in Figure 1.

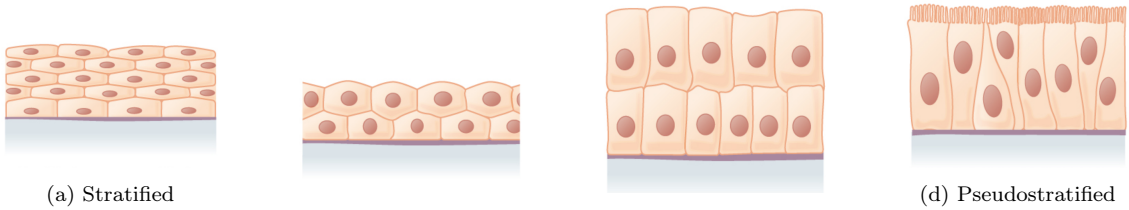


(a) Squamous (b) Cuboidal (c) Columnar

Figure 1: Simple epithelium [13]

Epithelial cells have also been observed in multilayer, or stratified, structures. Figure 2 shows these formations. In these cases, the first layer is denoted as the 'basal' and those above are referred to as 'suprabasal'. Our model must therefore investigate

the conditions under which cell proliferation can be controlled to accurately model, regulate and produce these structures in CC3D.



(a) Stratified squamous (b) Stratified cuboidal (c) Stratified columnar (d) Pseudostratified columnar

Figure 2: Stratified epithelium [13]

2.2 Cellular Dynamics & Cell Proliferation

Cell proliferation is the process in which cells grow and subsequently divide into two daughter cells, shown in Figure 3. The rate at which this occurs is governed by internal cellular mechanics and intra-cellular dynamics. To define a model for collective cell formation, we must first examine these mechanisms under which proliferation occurs. Among multiple environmental factors that can regulate proliferation, cell growth factors and cell-substrate adhesion are the most crucial [14, 15].

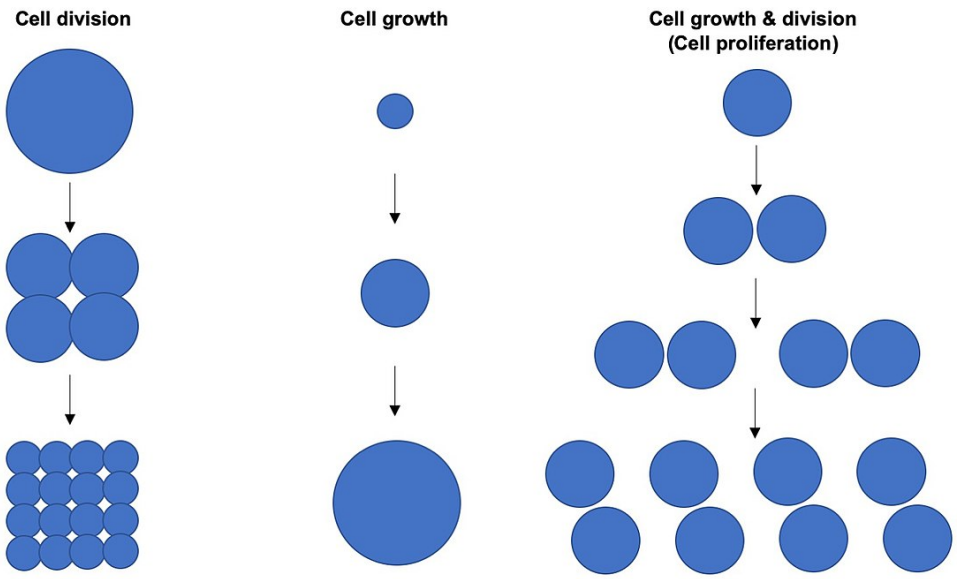


Figure 3: Cell proliferation [16].

2.2.1 Orientation of Division

During morphogenesis, oriented cell divisions are essential for the generation of cell diversity and for tissue shaping [17]. Yet, current evidence on the role of cell shape and different sets of intra-cellular mechanisms in orientating cell proliferation remains inconclusive [17, 18]. Therefore, it is essential for our model to include the effects orientation of cell division axis on varying mechanical properties of cells in various combinations.

2.2.2 Cell-Substrate Interactions

For many adherent cells, cell proliferation can only occur on a substrate [19], and cells have been shown to modulate their internal and external force generation in response to substrates or 3D cellular matrices [20]. Therefore, it is necessary to take into account cell-substrate interactions in our model.

Substrate stiffness has been experimentally demonstrated to increase the proliferation rate through contact inhibition via for the force of cell adhesion molecules, such as E-cadherin [21]. Cells sense the stiffness of substrate by generating forces that pull against the substrate, mediated by focal adhesions (as sites of integrin clustering) that form a physical link between the cell and the substrate [20]. The substrate maintains a dynamic force balance between the cell and its microenvironment, known as tensional homeostasis. Thus, the loss of the substrate or its relative stiffness can result in abnormal cellular behaviours, e.g. breast tumor progression [20]. Additionally, in an experiment studying the geometric control of cell life and death in human and bovine capillary endothelial cells, Chen et. al. [22] found that more adhesive substrates "promote growth by resisting contractile forces transmitted across integrins, thereby mechanically stabilising the nucleocytoskeletal lattice." They concluded however that cell shape governed the proliferation of individual cells, regardless of the type of adhesive integrin, indicating that the parameter

cell size has a greater effect on cell proliferation.

Doupé et al. [23] investigate the lineage of basal (first layer) cells and test the hypothesis that multi-cell units are maintained by central stem cells and their clones. They conclude that basal cells produce clones of irregular geometries where cell fate and cycle time was shown to be random. This has significant relevance to our model, as in Section 3.2.2, our model's initial condition is defined based on a single cell which proliferates dependent on parameter inputs.

2.2.3 Cell Compressibility

Proliferation has also been shown to increase with cell size (i.e. the cross-sectional cell area), notably in an experiment by Streichan et al. [24]. Here, they studied the response of applied force on model mammalian epithelium and experimentally observed increases in cell area preceded fraction of cycling cells (FCC), or division through investigations of model tissues on substrates. They concluded that proliferation rate increases with cell and growth rate. This conclusion was also supported by Jorgensen & Tyers in an investigation into the link between cell size and the number of cells inducing growth and division through genetic screens in *Drosophila*, fruit fly salivary glands.

While this has been experimentally observed, the mechanisms for cell growth and size themselves remain largely understudied. The internal mechanisms that govern individual cell compressibility are highly complex and hence mathematical modelling is the ideal tool to theorise these results. Mathematical modelling has previously been used to analyse the effect of cell size and growth with proliferation rates in comparison to physical cell distributions of lymphoblasts [25]. Results indicated that mammalian cells have internal mechanisms that regulate cell compressibility and hence maintain cell size. This supports the conclusion that proliferation rate is size-dependent.

It has also been shown that the cell area (and thus the proliferation rate) enhances as cells sense available space, using purely mechanical manipulation of epithelial tissues [24, 26] In the paper "Different responses to cell crowding determine the clonal fitness of p53 and Notch inhibiting mutations in squamous epithelia" by Kostiou et. al. epithelial tissues are modelled using 3 types of cells, namely basal layer progenitor, basal differentiating and suprabasal [8]. The focus of their investigation was spatial competition between mutated and fitter cell populations. They observed that proliferation rate of the tissue is limited by space restrictions and dependent on feedback between neighbouring cells, where the model balances cell density. Hence these feedback mechanisms provide a model for cell dynamics based on spatial regulatory limitations. Therefore, the spatial constraints of our model will be particularly useful for proliferation manipulation.

2.2.4 Cell-Cell Interactions

In cellular biology, it has been shown that cell shape and growth has the capacity to regulate cell proliferation, but only recently has this been linked to internal and external forces applied on single cells [REF]. Cells 'pull' on the extracellular matrix (ECM) and adjacent cells (cell-cell interactions) to determine the stiffness of their microenvironment [20]. This is due to the same intra-cellular force-generating protein as cell-substrate interactions, integrins. Numerical simulations have also investigated the link between cell-cell interaction and packing geometry. Farhadifar et al. [27] use a vertex model to generate different network morphologies and compare this to experimental observations. They found that junctional force balances were an essential driver for cell proliferation. Therefore, we must take into account these interactions as an important variable in our collective cell modelling.

In relation to epithelial cells structures, columnar cells have been show to internally increase their E-cadherin forces so as to adhere more strongly to each other [28] and down-regulate their expression of matrix proteins, decreasing their cell-substrate

adhesion [7,29,30]. In contrast, squamous cells have been shown to decrease their E-cadherin expression [30,31] and increase their cell-matrix, or cell-substrate adhesion [7].

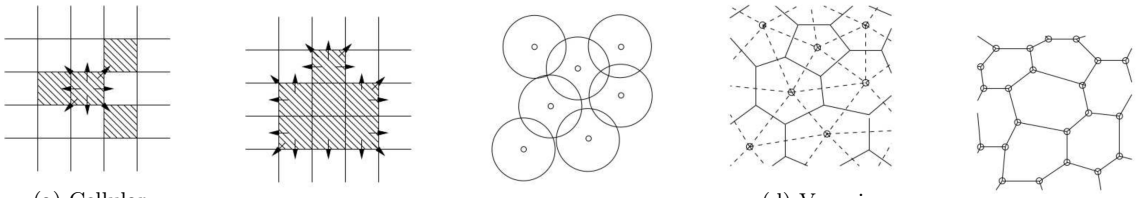
2.2.5 Cell Cortex Contractility

Cortex contractility represents the contractile tension along cell boundaries. In previous computational cell investigations, increasing the strength of this contractility limits cell growth and motility [9] and in fact has a a significantly stronger effect on cell structural regimes than cell-adhesion effects. Farhadifar's vertex model investigates the link between the junctional forces due to cell cortical contractility and proliferation [27]. This is due to actomyosin complex that forms within the cytoskeleton, i.e. the cell lattice. This actomyosin contractility has been shown to affect cell shape and adhesion [9] as increased contractility limits cell migration, and hence proliferation. Cell's internal modulation of actomyosin contractility affects the morphology of the junctional force network in epithelial sheets and also has significant effects on cell shape and tissue formation [32–34]. Further research in this area is needed to better understand the strength of this relationship. Hence, cell cortex contractility will form a vital part of our model as a penalty on cell growth and proliferation.

2.3 Mathematical Modelling

Mathematical modelling is a particularly useful means of inquiry as it involves the quantitative physical modelling of biological processes and bodies through force and mechanical analysis. Mathematical and subsequent computational models are ideal tools to analyse how multicellular interactions and intra-cellular dynamics drive emergent phenomena of pattern formation in tissues [35]. In general, such theoretical models are categorised as on- and off-lattice models. Five potential models for

cellular analysis are shown in Figure 4 and described below [36].



(a) Cellular Automaton Model (CAM) (b) Cellular Potts Model (c) Overlapping Spheres Model (d) Vornoi Tesselation Model (VTM) (e) Vertex Model

Figure 4: Cell-Based On- and Off-Lattice Model Schematics [36]

2.3.1 On-Lattice Models

On-lattice models constrain cells to lie on an artificial lattice. Figure 4(a) shows the Cellular Automata Model (CAM). This is the computationally simplest, allowing vast cell simulations, with each lattice containing no more than a single cell. Cell behaviour is time-state dependent, modelled from deterministic and stochastic rules. Figure 4(b) shows the Cellular Potts Model. In this case, multiple lattice sites are used to create more complex cell shapes, while still maintaining a level of computational ease. This is the model that this project will utilise, with more detail is given in Section 3.1.

2.3.2 Off-Lattice Models

In the cases of off-lattice models, cells are represented by point-mass particles based on Newtonian equations of motion [37], rather than constrained to a lattice. This allows complex regimes and migration but significantly increases computational complexity. Figure 4(c) shows the Overlapping Spheres Model (OSM), which tracks the movement of the centre of quasi-spherical cells. In this case, interaction is not directly defined but rather cells deform upon cell-cell or cell-substrate contact. The Voronoi Tesselation Model (VTM), Figure 4(d), is similar to OSM but Delaunay triangulation is used to model neighbouring cells, based on face contact. Finally, the Vertex Model shown in Figure 4(e) models cells as polygons, with the driving

factor being cell membrane behaviour. The vertices, in contrast to the centre, of each cell moves dependent on internal forces, including compressibility, contractility and adhesion.

3 Methodology

The project approach is broken up into three stages. Firstly we utilise current theoretical modelling approaches in cellular biology, that take into account the factors responsible for cell proliferation, as discussed in Section 2.2. Following this, we apply these models in computational simulations to produce collective cell structures. Finally, we propose simple numerical models based on these simulations to assess the validity of our results.

3.1 Theoretical Modelling: Cellular Potts Model

Our model simulates the 3D structure of tissue cells by focusing on the role of the mechanical properties of cells and cell-cell interactions. This is represented using the Cellular Potts Model (CPM). Also known as the Glazier-Graner-Hogeweg model (GGH), this was developed by François Graner and James Glazier to simulate cell sorting [38, 39] and was subsequently popularised by Paulien Hogeweg for studying morphogenesis [40].The CPM is a lattice model which is computationally and conceptionally simpler than most off-lattice models (e.g., vertex model), while it provides realistic descriptions of cell shapes [36, 41].

The CPM represents the cells on a two-dimensional lattice, where each cell covers a set of connected lattice sites or pixels, where each pixel can only occupied by one cell at a time. In this project, the lattice is a rectangular surface (480 x 195 pixels in the x- and y-dimensions, respectively), representing a cross-sectional view to epithelial cells placed on a substrate. The expansion and retraction of the cell boundaries

are determined by minimising a phenomenological Potts energy E, defined in terms of the area A_σ and perimeter L_σ of each cell σ of N cells (indices $\sigma = 1, ..., N$) [9, 27, 42–44] as:

$$E = \lambda_{\text{area}} \sum_{\sigma}^{N} (A_\sigma - A_0)^2 + \lambda_{\text{cont}} \sum_{\sigma}^{N} L_\sigma^2 + \sum_{\vec{i},\vec{j}} J\left(\sigma_{\vec{i}}, \sigma_{\vec{j}}\right)\left(1 - \delta(\sigma_{\vec{i}}, \sigma_{\vec{j}})\right) \quad (1)$$

Here, the prefactors λ_{area}, λ_{cont} and λ_{adh} reflect the relative importance of the corresponding cellular properties to set cellular morphology.

The first part of this equation represents the inherent energy due to cell size. This models the compressibility of cells by penalising the deviation of cell areas from target area A_0. Hence, energy is minimised when individual cell areas are closest to the ideal target area.

The second part of the equation is based on cell membrane tension, which is defined based on the cortex contractility coefficient λ_{cont}, where cell cortex is modelled as a spring with a target zero equilibrium length (i.e. the target length of the cell perimeter is zero).

The final part of the equation captures the energy arising from cell-cell and cell-substrate adhesion due to the forces of adhesion molecules such as E-cadherin [6]. The Kronecker delta, δ is included here to prevent pixels inhibiting the same cell to be counted. $J\left(\sigma_{\vec{i}}, \sigma_{\vec{j}}\right)$ gives the boundary energy cost at neighbouring lattice sites $\vec{i}$ and $\vec{j}$. When both lattice sites correspond to cells, $J(\sigma_{\vec{i}}, \sigma_{\vec{j}}) = \lambda_{\text{adh}_{\text{cc}}}$. When one lattice site corresponds to a cell and the other site corresponds to the substrate, $J(\sigma_{\vec{i}}, \sigma_{\vec{j}}) = \lambda_{\text{adh}_{\text{cs}}}$. Otherwise when one or both lattice sites represent empty space or the boundary wall, J is equal to 0. Note that $\lambda_{\text{adhesion}} < 0$ to represent that cells preferentially expand their boundaries shared with neighbouring cells. This is however balanced by the contractile tension along the cell cortex.

3.2 Implementation: CompuCell3D Software Package

The CPM model is implemented using an open-source software package called CompuCell3D, developed by Prof. James Glazier (of Glazier-Graner-Hogeweg) [45]. The computational simulations produced from this form the main method of analysis for this project.

Each lattice site is based on a defined number of pixels, and cells are ideally modelled hexagonally to mimic realistic cell shapes. Cells are reproduced based on the given conditions on environment size and other variables as shown in Equation (1).

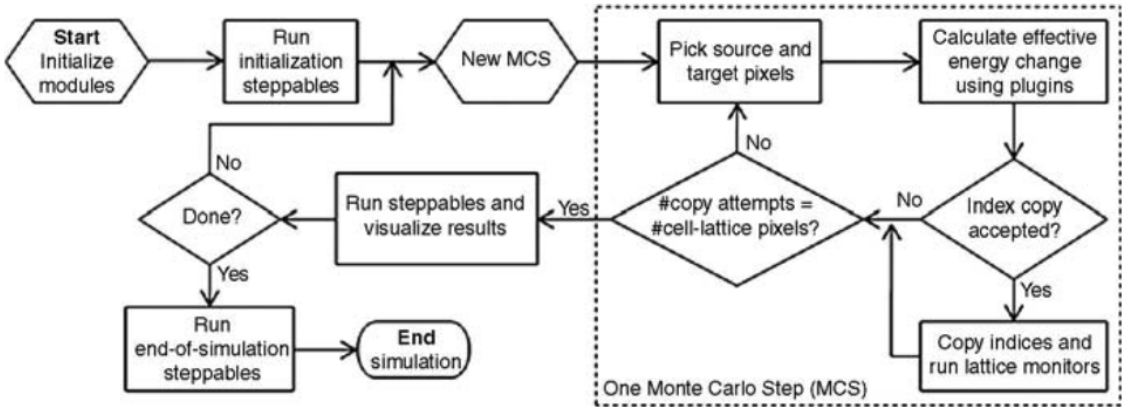


Figure 5: Flow chart of the GGH algorithm as implemented in CompuCell3D [45].

The program simulates cell proliferation through time via Monte Carlo Steps (MCS), the natural time unit of the model, defined as N index-copy attempts, where N is the number of sites in the cell lattice. A Monte Carlo step is set to l elementary steps – where l is the total number of lattice sites in the simulated area [45]. Figure 5 shows how CC3D operates an algorithm as a stochastic series of elementary steps. The dynamics of the CPM is defined by a stochastic series of elementary steps, where a cell expands or shrinks accommodated by a corresponding area change in the adjacent cell (or empty area) [39,45]. The adjacent lattice sites, $\vec{i}$ and $\vec{j}$ occupied by different cells $\sigma_{\vec{i}} \neq \sigma_{\vec{j}}$, are randomly selected by the algorithm. The elementary step is an attempt to copy $\sigma_{\vec{i}}$ into the adjacent lattice site $\vec{j}$, which takes place with

probability:

$$P\left(\sigma_{\vec{i}} \to \sigma_{\vec{j}}\right) = \begin{cases} 1 & \text{for } \Delta E \leq 0 \\ e^{-\Delta E/T} & \text{for } \Delta E > 0 \end{cases} \tag{2}$$

where ΔE is the change in functional energy due to the elementary step considered, defined in Equation (1). The temperature parameter T is an arbitrary scaling factor. CompuCell3D is available from: http://https://compucell3d.org/.

Together, Equations (1, 2) imply that cell configurations which increase the penalties in Equation (1) are less likely to occur. Thus, the cell population evolves through stochastic rearrangements in accordance with the biological dynamics incorporated into the effective energy function E.

3.2.1 Probability of Cell Division

Based on the experimental observations of cell proliferation discussed in Section 2.2, the probability of division has been defined in the form of the Hill function. The Hill function has previously been used in biological investigations into binding of molecular cell structures [46]. At every MCS, if a cell σ reaches its target area (i.e., $A_\sigma \geq A_0$), the cell may proliferate with probability of the form:

$$P_{\text{div}} = P_{\text{max}} \frac{n_s^k}{n_s^k + \left(\gamma\sqrt{A_0}\right)^k} \tag{3}$$

where P_{max} is the maximum probability of proliferation and n_s denotes the number of boundary pixels of a cell adjacent to the substrate. For simplicity, we assume that the Hill half-saturation threshold is given by the dimension in pixels of a square shaped cell, $\sqrt{A_0}$. γ is a multiplicative factor of the initial value of n_s, representing the 'half-saturation constant'. k denotes the Hill coefficient, which in our simulations

is set to 10. This equation therefore shows, in line with experimental observations, that proliferation rate increases with cell-substrate adhesion. This is justified by the experimental observations [22] that prove that increased cell spreading on substrate increases the area of cell substrate contacts and leads to cell growth, and thus proliferation. Recall that, following [19], cells not attached to the substrate do not proliferate.

If a division occurs, a cell is divided along a plane specified by a normal vector $n_{\text{div}} = (n_x, n_y, n_z)$ where n_x, n_y and n_z are the components normal to the plane. In our case, our model consists of a 2D lattice so we are only examining horizontal and vertical cell division, hence $n_{\text{div}} = (n_x, n_y)$. We will consider x-orientation, y-orientation and random orientations of proliferation, since it remains elusive how the orientation of cell proliferation is influenced by cell shape and mechanical forces, due to cellular interactions [17].

Cell division results in a parent cell (which existed before the division) and a child cell (a new cell), each with area $\approx A_0/2$. These cells then grow to reach the target area A_0, with the rate of growth dependent on the relative strengths of the compressibility and contractility parameters, λ_{area} and λ_{cont} given in Equation 1.

3.2.2 Initial Conditions

The figure below demonstrates the initial condition for running simulations in CC3D for collective cell formation analysis. A single cell with a size of 15 x 15 pixels is placed on a substrate, with an empty region of 500 x 500 pixels to proliferate. CC3D allows the wall cells (each 10 x 10 pixels) to be excluded from the proliferation interaction of the Potts model [45] through CC3D's built-in 'Freeze' function.

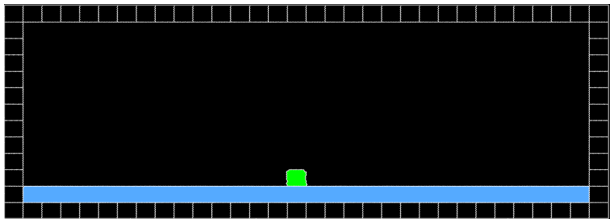


Figure 6: Initial condition. A single cell (green) with a size of 15 × 15 pixels is placed on a substrate (blue) and faces an empty region (black) surrounded by the wall cells (black squares).

Table 1 summarises the final parameter values used in the computational simulations, adapted from [9].

Table 1: Model parameters.

Parameter	**Value**
Initial cell size (pixel × pixel)	15×15
Initial cell area, A_σ (pixel × pixel)	225
Preferred area, A_0 (pixel × pixel)	225
Temperature, T	50
Half-saturation constant, γ	2
Hill coefficient, k	10
Maximal proliferation probability, $P_{\max}$	0.1

4 Results & Discussion

In this section we present the overall project results, including simulations, numerical analysis and discussion, for the cases of cell proliferation both disabled (single cell morphology) and enabled (multicellular morphology).

4.1 Single Cell Morphology

Since multicellular morphogenesis is partly driven by changes in the shape of individual cells [47], the starting point in our investigation was to explore single-cell morphology in response to its mechanical properties. In these simulations, cell proliferation is disabled.

4.1.1 Computational Analysis

Typical snapshots of single-cell morphology in the steady-state are shown in Fig. 7. These phase diagrams give the sensitivity analysis of our results by examining the effects of each of our proliferation parameters, holding all else constant. It is evident that increasing cell-substrate adhesion produces squamous (i.e. flat) cell shapes, yet increased contractility counteracts this and promotes cuboidal cell shapes, as we can see in the top right morphology of each of Figure 7(a, b).

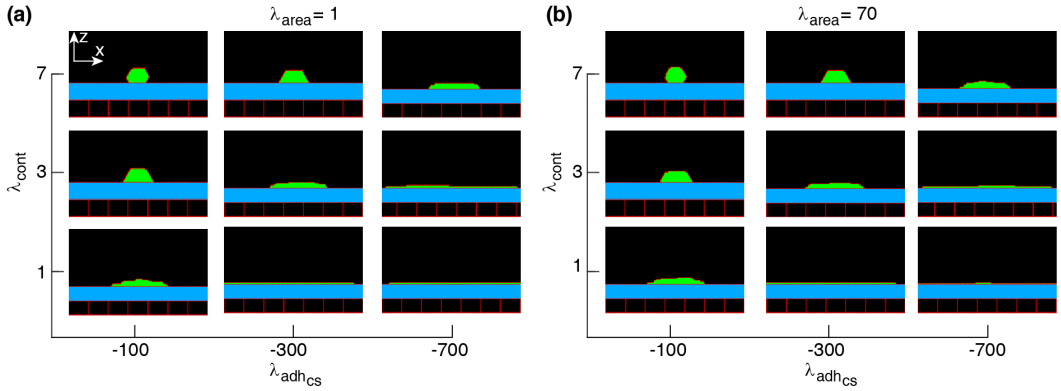


Figure 7: Phase diagram of single-cell shapes. x-z cross-section of the cell (green) placed on a substrate (blue) surrounded by an empty region (black) and wall cells (black squares). Simulations were run with the cell proliferation disabled. The range of parameter values are adopted from [9]. Snapshots were taken in the steady-states.

To better understand how the single-cell morphology can influence multicellular dynamics, we analyse cell area and the number of cell-substrate adhesion pixels, which affects the probability of cell proliferation, in response to the mechanical control parameters. This enables us to predict the combination of mechanical properties that can lead to different collective cell behaviours and formations.

Figure 8(a) shows that the average cell area increases with cell-substrate adhesion, which is more evident with weak contractility. Contrarily, with strengthening cell contractility and decreasing cell-substrate adhesion the average area of a cell falls below its target area A_0. On the other hand, with increasing λ_{area}, the average cell area remains close to A_0 at all λ_{cont} and $\lambda_{\text{adh}_{\text{cs}}}$; see Figure 7(b). Additionally, increasing cell-substrate adhesion and weakening cell contractility expands cell-substrate adhesion sites, see Figures 7(c, d).

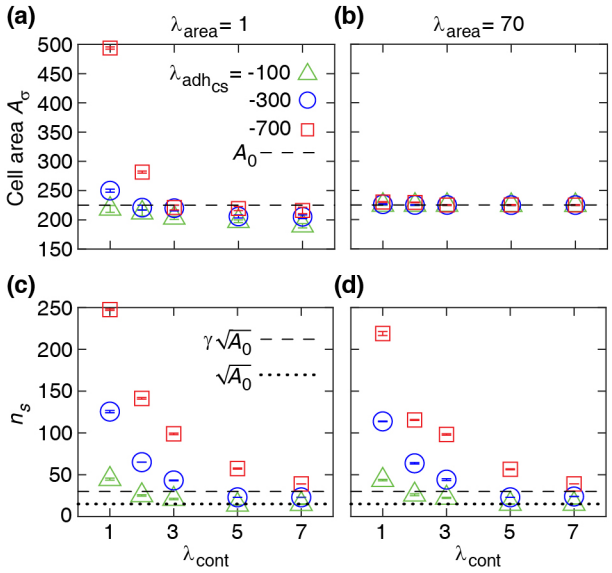


Figure 8: CPM simulation results for single-cell properties. (a, b) Area of the cell in steady state versus contractility strength λ_{cont} at various λ_{area} and cell-substrate adhesion $\lambda_{\text{adh}_{\text{cs}}}$. (c, d) Number of cell-substrate adhesion sites n_s in the steady state versus λ_{cont} at various λ_{area} and $\lambda_{\text{adh}_{\text{cs}}}$. Each symbol is derived from an individual simulation run and corresponds to mean ± SD.

4.1.2 Numerical Analysis & Discussion

To assess the consistency of the computational simulation results we estimate the energy minimum for a simplified rectangular cell shape. The energy function (from Eq. 1) for a single rectangular cell is written:

$$E(l,h) = \lambda_{\text{area}} \left(lh - A_0\right)^2 + \lambda_{\text{cont}} (2l + 2h)^2 + \lambda_{\text{adh}_{\text{cs}}} l \tag{4}$$

where l and h are cell length and height (see schematic inset, Fig. 9). The minimum of this energy E is determined by solving the following equations:

$$\frac{\partial E(l,h)}{\partial l} = 0, \quad \frac{\partial E(l,h)}{\partial h} = 0 \tag{5}$$

which results in

$$\begin{aligned} 2\lambda_{\text{area}} h \left(lh - A_0\right) + 8\lambda_{\text{cont}} \left(l + h\right) + \lambda_{\text{adh}_{\text{cs}}} = 0 \\ 2\lambda_{\text{area}} l \left(lh - A_0\right) + 8\lambda_{\text{cont}} (l + w) = 0 \end{aligned} \tag{6}$$

to define cell length l^* and height h^* at mechanical equilibrium. It seems there is no general real solution for Equations 6. Hence, they are evaluated at various λ parameters and solved. This allows our numerical model to correspond to different cell morphologies. Figure 9 demonstrates the typical dependence of the energy function on the cell width and height over a range of λ parameters.

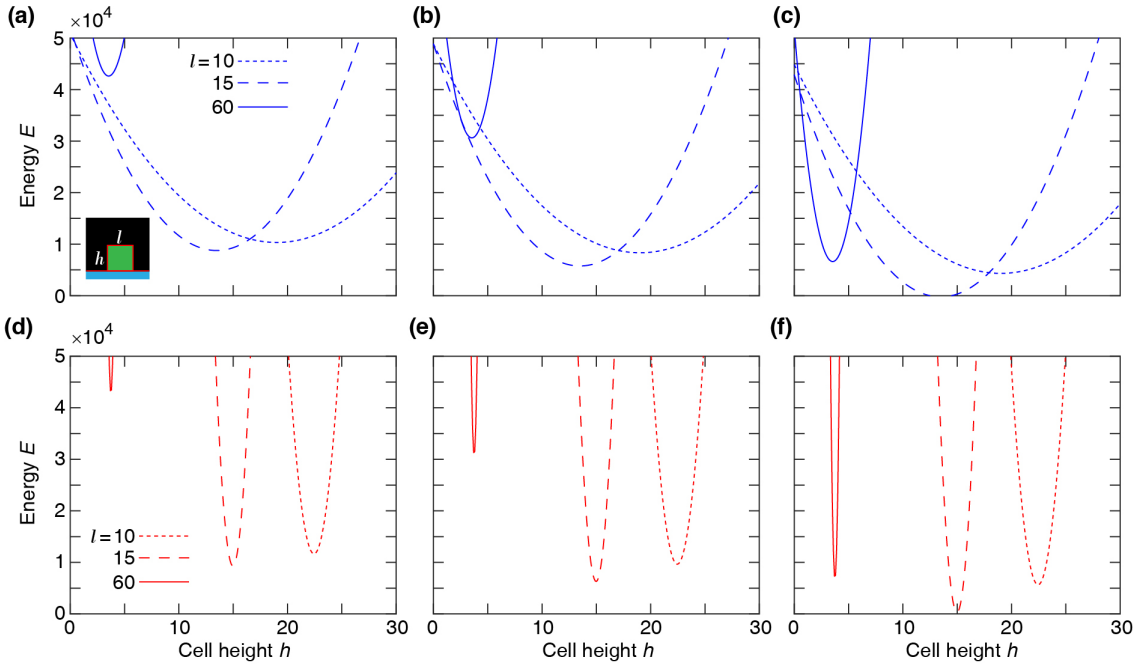


Figure 9: Single-cell Potts energy E versus cell height h at various cell length l calculated using Equation (4). Top row: $\lambda_{\text{area}} = 1$ and $\lambda_{\text{cont}} = 3$ at $\lambda_{\text{adh}_{\text{cs}}} = -100$ (a), -300 (b), -700 (c). Bottom row: $\lambda_{\text{area}} = 70$ and $\lambda_{\text{cont}} = 3$ at $\lambda_{\text{adh}_{\text{cs}}} = -100$ (d), -300 (e), -700 (f).

To further assess these numerical results, we examine individual cell aspect ratio based on this rectangular cell shape. We assume that mechanical equilibrium at steady state can be approximately estimated from the minimisation of the energy function corresponding to a rectangular cell and then calculate the relative cell aspect ratio and area. This is shown in Figure 10.

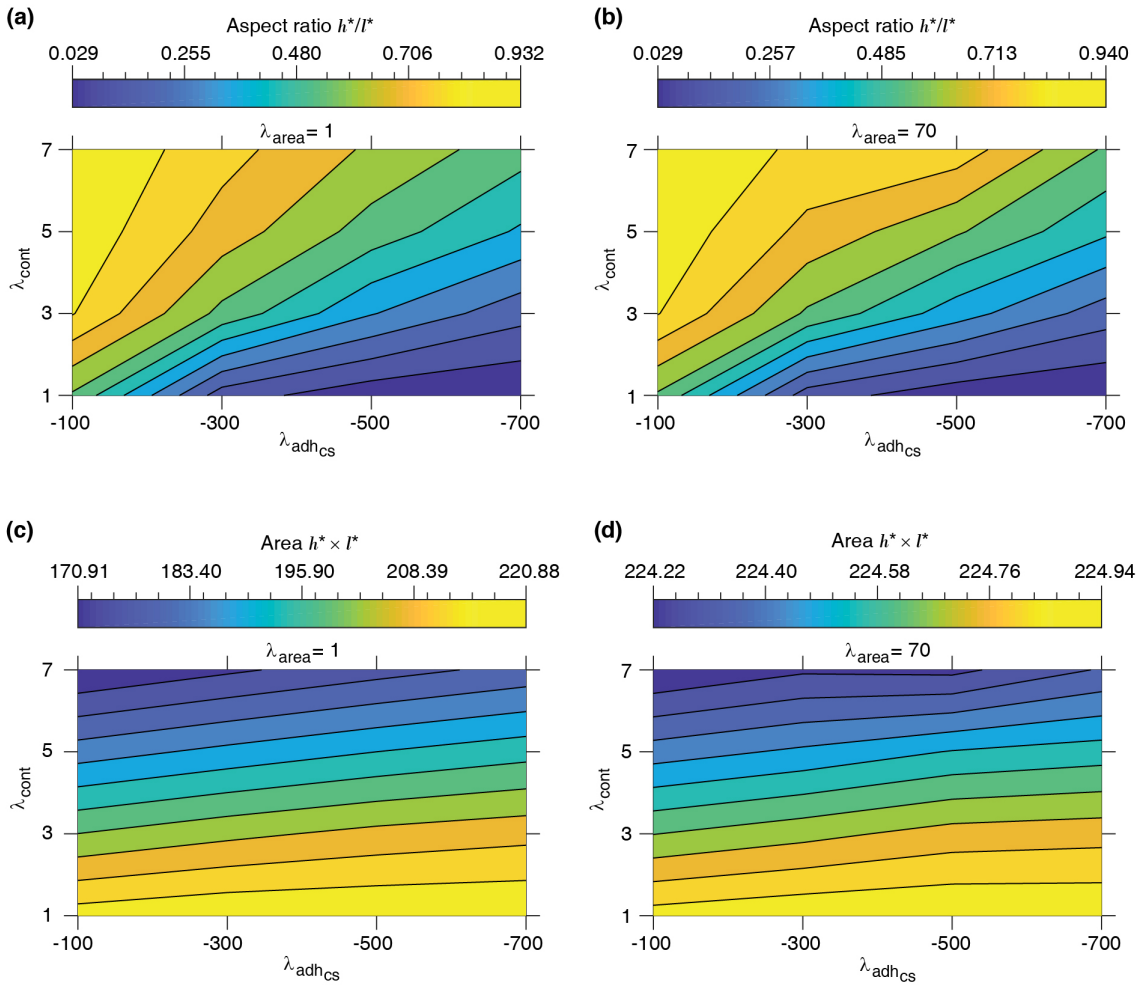
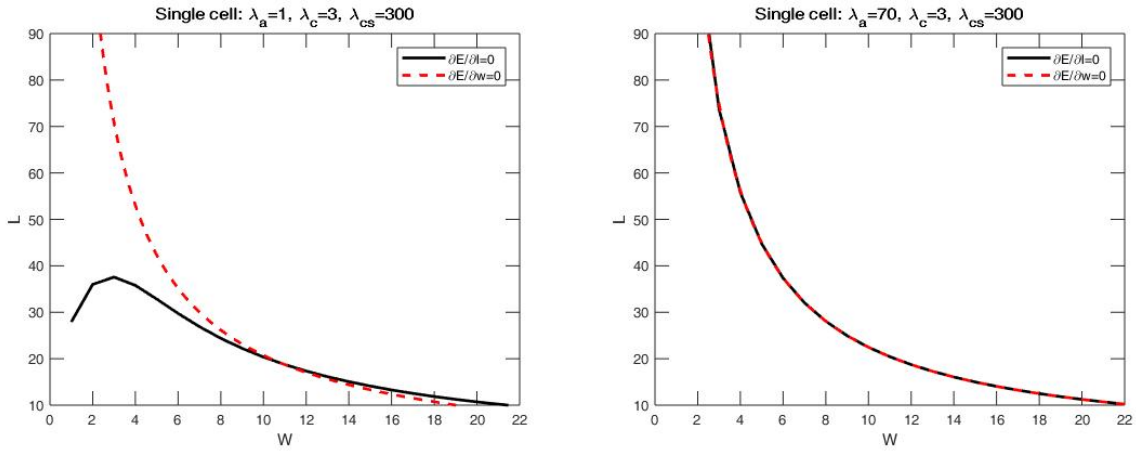


Figure 10: Equilibrium single-cell aspect ratio w^*/L^* and area $w^* \times L^*$ versus cell-substrate adhesion $\lambda_{\text{adh}_{\text{cs}}}$ and contractility λ_{cont}. l^* and h^*: cell length and height at the mechanical equilibrium, respectively, corresponding to rectangular cell shape; see Equation (4).

These phase diagrams are strongly consistent with the phase diagram of single-cell morphology shown in Figure 7. They also show qualititative agreement with the CPM simulation results in Figure 8(a, b).

To finally verify this rectangular cell model, we evaluate Equations 6 for various proliferation parameters, shown in Figure 11. Here, at high $\lambda_{\text{area}} = 70$ it seems (visually) there are more solutions. We can conclude from this that our model is more effective with larger cell size.



(a) (b)

Figure 11: Example solutions for Equations (6).

From these numerical results, we can make a number of conclusions on cell morphology that can be used to make predictions for our multicellular regimes. Together, these results suggest that monolayers and multilayered structures are more likely to form with increasing cell-substrate adhesion and weaker cell contractility, due to increased proliferation probability of individual cells. Furthermore, non-confluent structures are generated when cells have strong cortex contractility and low adhesion to the substrate.

An additional cell configuration of an ovular cell shape was also assessed, see Appendix A.

4.2 Multicellular Morphology

With cell proliferation enabled in the model, occurring at a rate denoted by Equation 2, we then studied the effect of the plane of cell division on multicellular formations using various combinations of mechanical parameters. As discussed in Section 3.2.1, our model allows a convenient way to simulate collective cell morphologies by considering different orientations of cell division axis through our normal division vector, $n_{\text{div}} = (n_x, n_y)$.

The simulations begin with a single cell placed on a substrate in the middle of the domain, and cell division occurs according to Equation 2, i.e. when the cell area is larger than A_0 with a probability dependent on the number of cell adhesion sites (pixels) attached to the substrate. We set proliferation to begin at MCS = 500.

Firstly, we examine when the plane of cell division is vertical (i.e. in X-orientation of division), i.e., $n_{\text{div}} = (1, 0)$. These simulations were run up to 40,000 MCS to reach a steady state, the snapshots of which are shown in Figure 12. In these cases, the daughter cell appeared on either the left or right of the original cell, which allowed confluent monolayers to form easily.

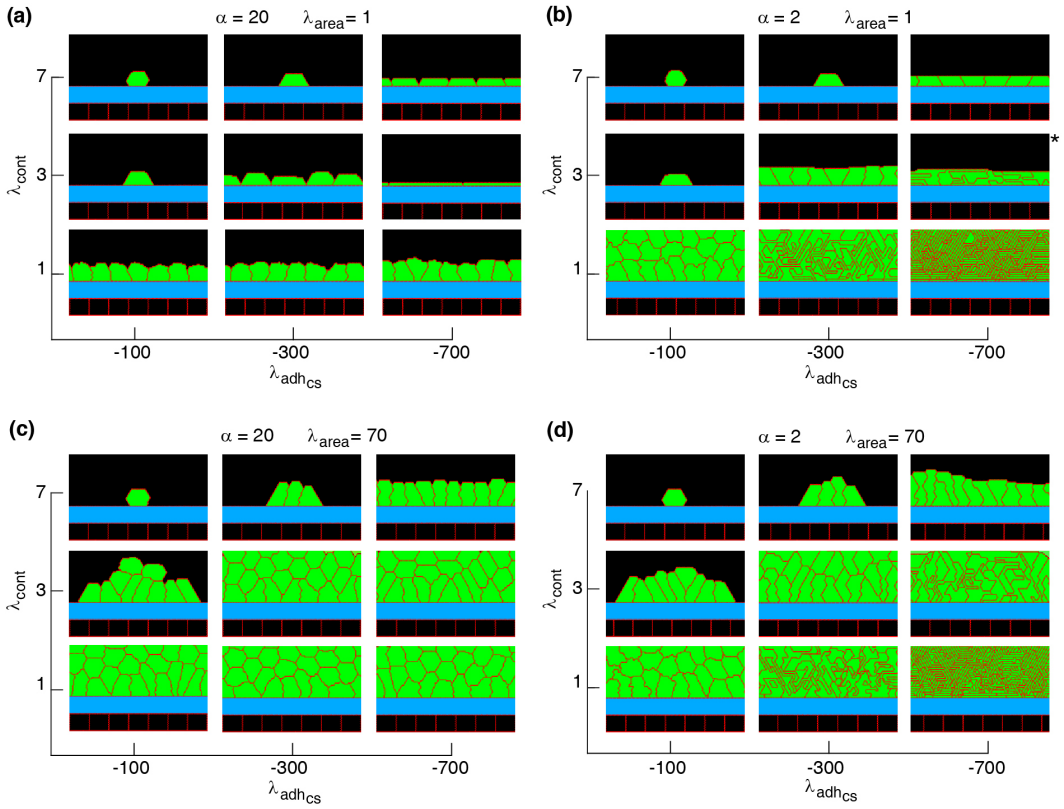


Figure 12: Phase diagram of steady state collective cell morphology when the axis of division is perpendicular to the substrate (vertical division), i.e., $n_{\text{div}} = (1, 0)$; see Equation (2). $\alpha = \lambda_{\text{adh}_{\text{cs}}}/\lambda_{\text{adh}_{\text{cc}}}$.

We repeated this process with a horizontal (i.e. in Y-orientation of division) division axis, where $n_{\text{div}} = (0, 1)$ shown in Figure 13. Here, daughter cells appeared on top of the original dividing cell. For confluent monolayers to form in this case, increasing cell-substrate adhesion was required. In the cases where multiple cells

are observed attached to the substrate, in the simulation these cells appeared to be 'pulled' down by the strength force of the substrate in an attempt to form more confluent structures. This was particularly evident with larger cell areas, i.e. when λ_{area} was increased.

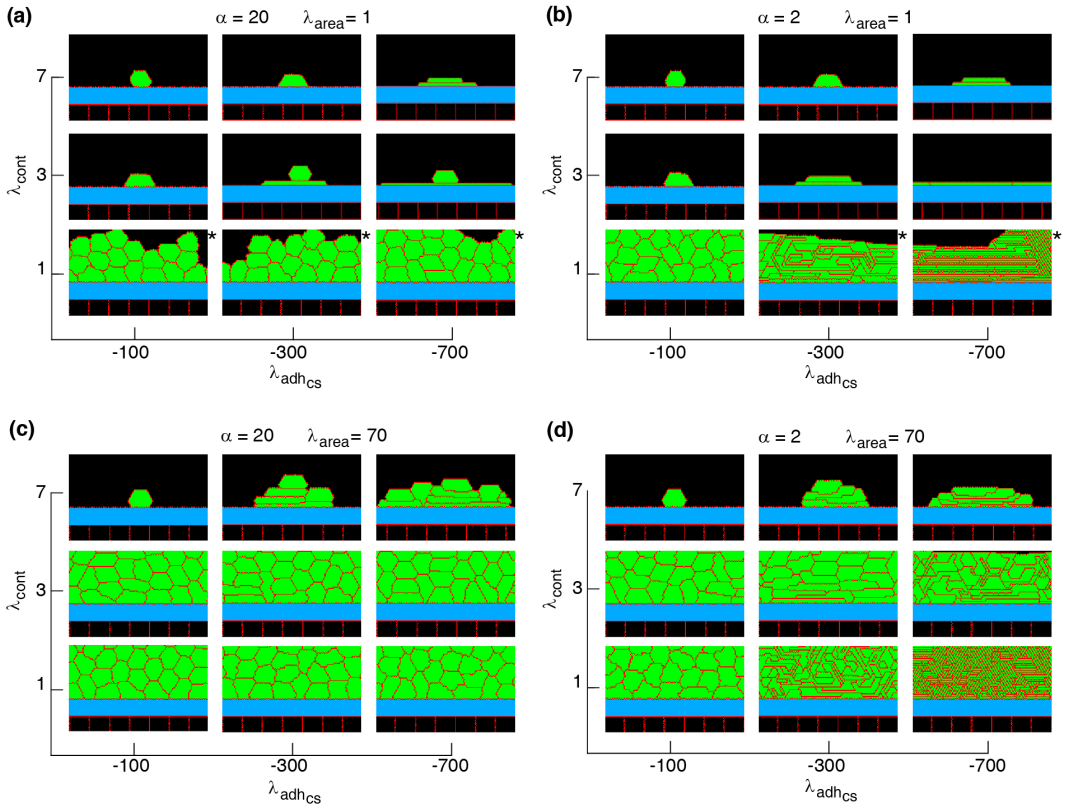


Figure 13: Phase diagram of steady state collective cell morphology when the axis of division is parallel to the substrate (horizontal division), i.e., $n_{\text{div}} = (0, 1)$, i.e., $n_{\text{div}} = (0, 1)$; see Equation (2). $\alpha = \lambda_{\text{adh}_{\text{cs}}}/\lambda_{\text{adh}_{\text{cc}}}$.

Finally, we set division axis to random, meaning that each time an individual cell divided, there was a random chance whether it would divide horizontally or vertically. The steady-state phase diagrams for these collective cell morphologies evaluated at various proliferation parameters is shown in Figure 14. Interestingly, this case followed the X-orientation of division cell structures more closely than that of Y-orientation of division.

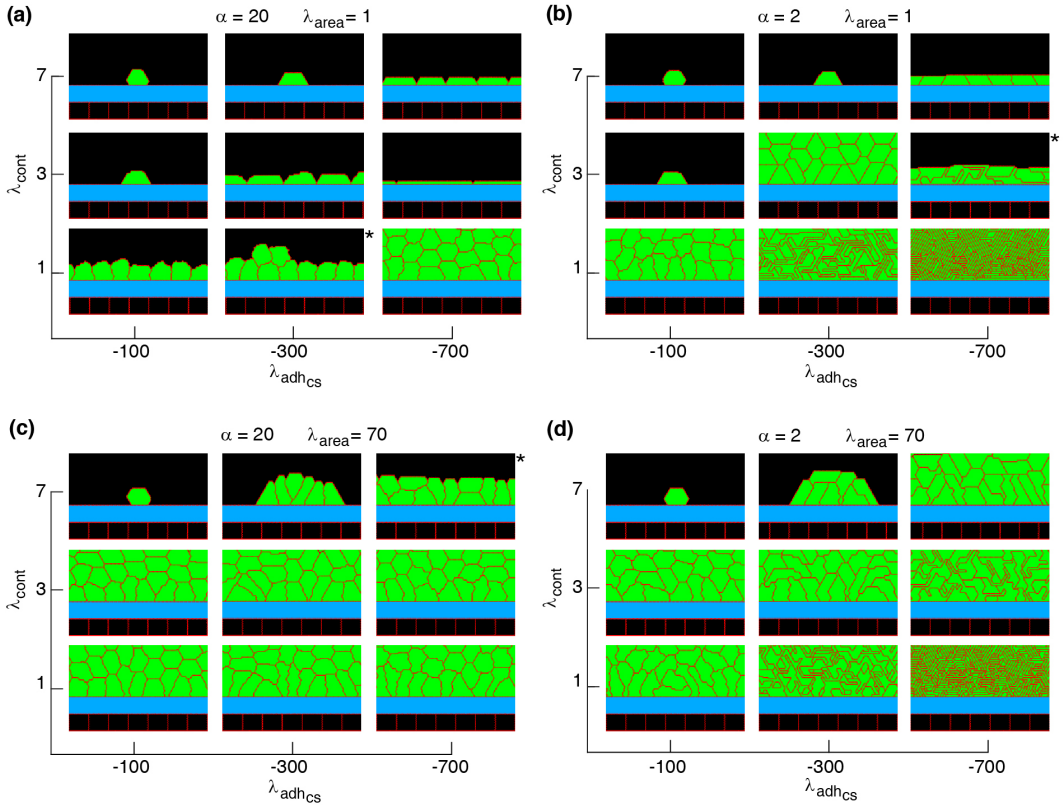


Figure 14: Phase diagram of collective cell morphologies with random orientation of cell proliferation. $\alpha = \lambda_{\mathrm{adh_{cs}}}/\lambda_{\mathrm{adh_{cc}}}$.

4.2.1 Discussion

First, it is clear that the cell shapes observed in the multicellular systems, in the majority of cases, are quite varied from the shape of a single isolated cell obtained for the same set of mechanical parameters. We can conclude that this is due to the introduction of the extracelluar dynamics resulting from the effect of cell-cell adhesion.

Three main types of multicellular structures and behaviours were observed over time in these simulations.

1. Firstly, for certain parameter combinations (i.e. strong cell cortex contractility λ_{cont}, low cell area strengthλ_{area} and low cell-substrate adhesion $\lambda_{\mathrm{adh_{cs}}}$), cell division is either completely blocked or severely limited. This resulted in the formation of a small group of cells that were unable to form a confluent cell

layer along the substrate over the whole domain. This was particularly evident with vertical cell division and high contractility.

2. A cell monolayer can form through repeated cell divisions in such a way that cell proliferation is formed. This layer may be composed of flat or tall cells and strongly follows the epithelial cell structures presented in Figure 1.

3. Thirdly, in multi-layered structures, cell division appears to continue indefinitely (although it is still restricted to the basal cells along the substrate) and the height of the cell layer increases over time. In a real multilayered epithelium, the height of such layer can be controlled by differentiation and death of the non-proliferating cells that are not adhered to the substrate. This could be implemented into our model by introducing cell death to incorporate the self-regulation experimentally observed of multilayered epithelial regimes.

In general, non-confluent structures are formed at high cell contractility and reduced cell-substrate adhesion, independently of the proliferation orientation; see Figures. 12-14. This is due to reductions in both cell area and the number of cell-substrate adhesion sites in individual cells which reduce the probability of cell proliferation, as seen in Figure 8.

Confluent monolayers and multilayered structures are formed with increasing cell-substrate adhesion and lowering cell contractility. With vertical proliferation orientation, monolayers of squamous (flat), cuboidal, and columnar (tall) cells are found; see Fig. 12. The expansion of monolayers typically happens through the division of border cells (at both edges of the monolayer) on the substrate, while the other cells inside the monolayer do not divide or only relatively rarely. Once the layer becomes confluent cell crowding limits the cell area and the cell-substrate adhesion sites, due to which the probability of proliferation decreases; see Fig. 15(a-i). At certain combinations of mechanical properties (summarised in Fig. 12 and analysed in Fig. 15(a-i), cells stop proliferating once a confluent monolayer is formed after

they hit the boundary walls, resulting in the formation of monolayers. Squamous cells are mostly found for high cell-substrate adhesion (relative to cell-cell adhesion), increased cortex contractility and reduced λ_{area} parameter. Increasing λ_{area}, in this regime, leads to squamous-to-columnar shape transition.

A major factor that contributes to monolayer-to-multilayer transition is cell crowding. When the cell density cannot increase anymore in the basal layer, while cell deformations are likely, cells are extruded from the monolayer. Accordingly, cells at the basal layer can expand their area increasing the probability of their proliferation and further extrusion events; see Fig. 15(j-l). Overall, monolayer-to-multilayer transition is more likely to appear with a combination of parameters that increase λ_{area}, increasing cell-substrate adhesion, reduced cortical contractility, and with random proliferation orientation or being parallel to the substrate; see Figures. 13 and 14.

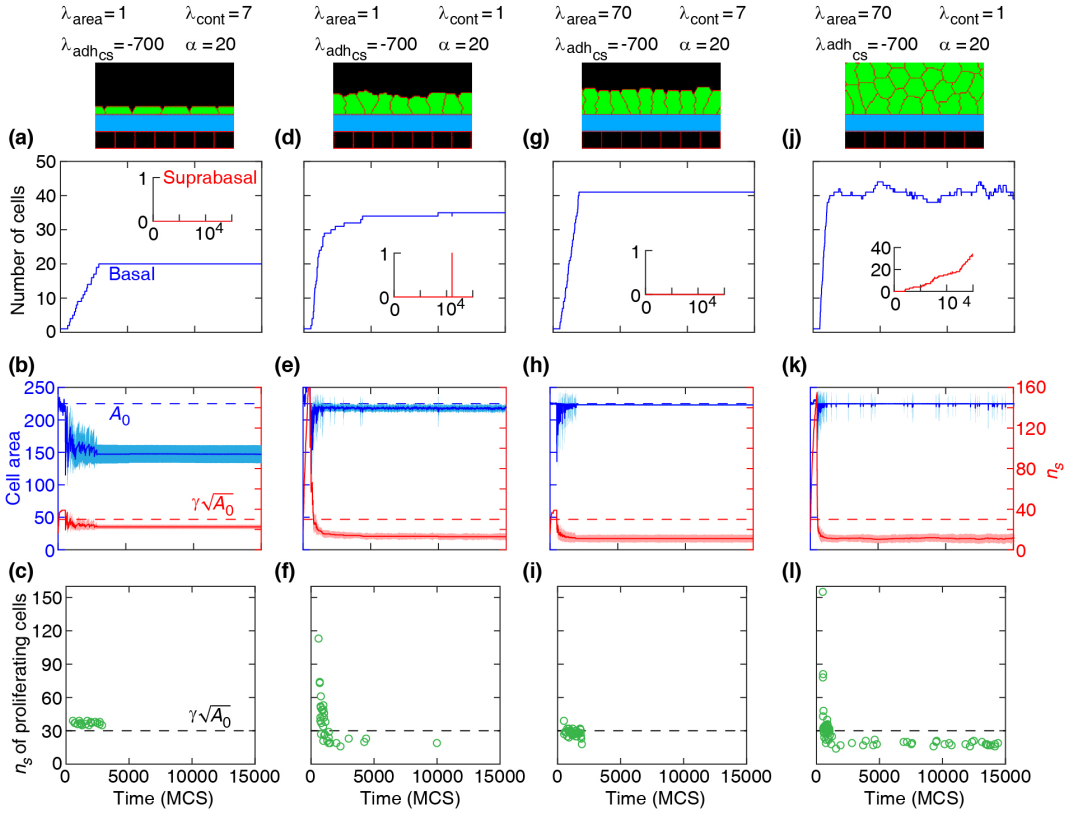


Figure 15: Dynamics of basal cells in four different collective morphologies illustrated with snapshots and associated mechanical parameters. Top row: number of basal cells (with adherence to the substrate) and suprabasal cells (without adherence to the substrate) versus time. Middle row: area (left axis) and number of cell-substrate adhesion sites n_s (right axis) for basal cells versus time. Solid curves: mean. Shaded region: SD. Bottom row: n_s of proliferating cells versus time. Orientation of cell proliferation is vertical, $n_{\rm div} = (1, 0)$. $\alpha = \lambda_{\rm adh_{cs}}/\lambda_{\rm adh_{cc}}$.

From these simulations, the area strength parameter, $\lambda_{\rm area}$, was identified as a major factor that influenes collective morphology. Increasing $\lambda_{\rm area}$ generates multilayer structures, whereas with reducing $\lambda_{\rm area}$ monolayer and non-confluent structures appear, see Figures 12-14. Increasing this parameter increases the likelihood of proliferation by accelerating cell area to grow and approach the target area A_0. When $\lambda_{\rm area}$ is lower, the probability of proliferation is decreased as cell area deviates from A_0. This effect is most evident in the generation of monolayers, where cell proliferation is limited by cell area and the number of cell-susbtrate adhesion sites; see Fig. 15(a-i). Increasing cell-substrate adhesion and weakening cell cortex contractility expands the number of cell-substrate adhesion sites.

Meanwhile, in multilayered structures, the growth of cell area is facilitated such that the crowding does not block cell proliferation events. Here, continuous cell extrusions out of the basal layer lead to multilayered structures, see Fig. 15(j-l). At a stronger cell-substrate adhesion however, the extruded cells may retun back to the basal layer, see Figure 15(d).

The validity and consistency of our multilayered simulations can be assessed by comparing previous experimental observations. Our simulation results complement earlier findings and are consistent with experimental observations. It has been observed that the probability of cell proliferation increases with cell area [24] and reduction in cell area (imposed by mechanical constraints on tissue expansion) inhibits cell proliferation [22, 26]. Further, substrate stiffness has been known to be positively correlated with cell proliferation increasing substrate stiffness (dependent on cell-substrate adhesion) and was found to increase the proliferation rate [20, 21, 48]. It was shown that when the cell density cannot increase anymore in a monolayer (due to cell crowding), while the proliferation events still occur, newly generated cells are extruded out of the monolayer where they remained without adhering to the substrate [3]. These suprabasal cells may slide over the basal cells such that they migrate in and become basal cells themselves [49, 50].

The simulations of cell shapes in monolayers are also consistent with experimental observations, noticeably with vertical cell division ($n_{\text{div}} = (1, 0)$. For example, with intermediate cortical contractility $\lambda_{\text{cont}} = 3$ and $\lambda_{\text{adh}_{\text{cs}}} = -300$, increasing adhesion ratio α from 2 to 20 reduces cell height by factor 1.88 (from 15.44 to 8.23). This can be seen visually by comparing the middle snapshots in Figure 12 (a, b). For squamous cells ($\lambda_{\text{cont}} = 7$ and $\lambda_{\text{adh}_{\text{cs}}} = -700$), cell height drops by factor 1.22 (from 8.23 to 6.75), as shown in the top right corner snapshots in each Figure 12 (a) and (b). For columnar cells, a negligible reduction (by factor 1.10, from 22.35 to 20.32) in cell height is found with increasing α; compare top right corner snapshots in Fig. 12(c, d). These simulation results agree with experimental observations

that lowering the lateral cell–cell adhesion decreases cell height [7, 30, 31, 51]. It is also consistent with the theoretical prediction that the cell-cell lateral adhesion is a crucial parameter to increase cell height [52, 53]. Further, the thin multicellular shapes, formed at high adhesion and low contractility, are consistent with the so called "soft" cell shape regime [9, 27, 42].

We can further define the difference between squamous and tall epithelial cells within a monolayer by assessing a range of factors over the simulations. Figures 16 & 17 demonstrate the number of cells in relation to when proliferation occurs. They also show average cell area, eccentricity and height (vertical COM) as a means of comparing cell shapes in these regimes.

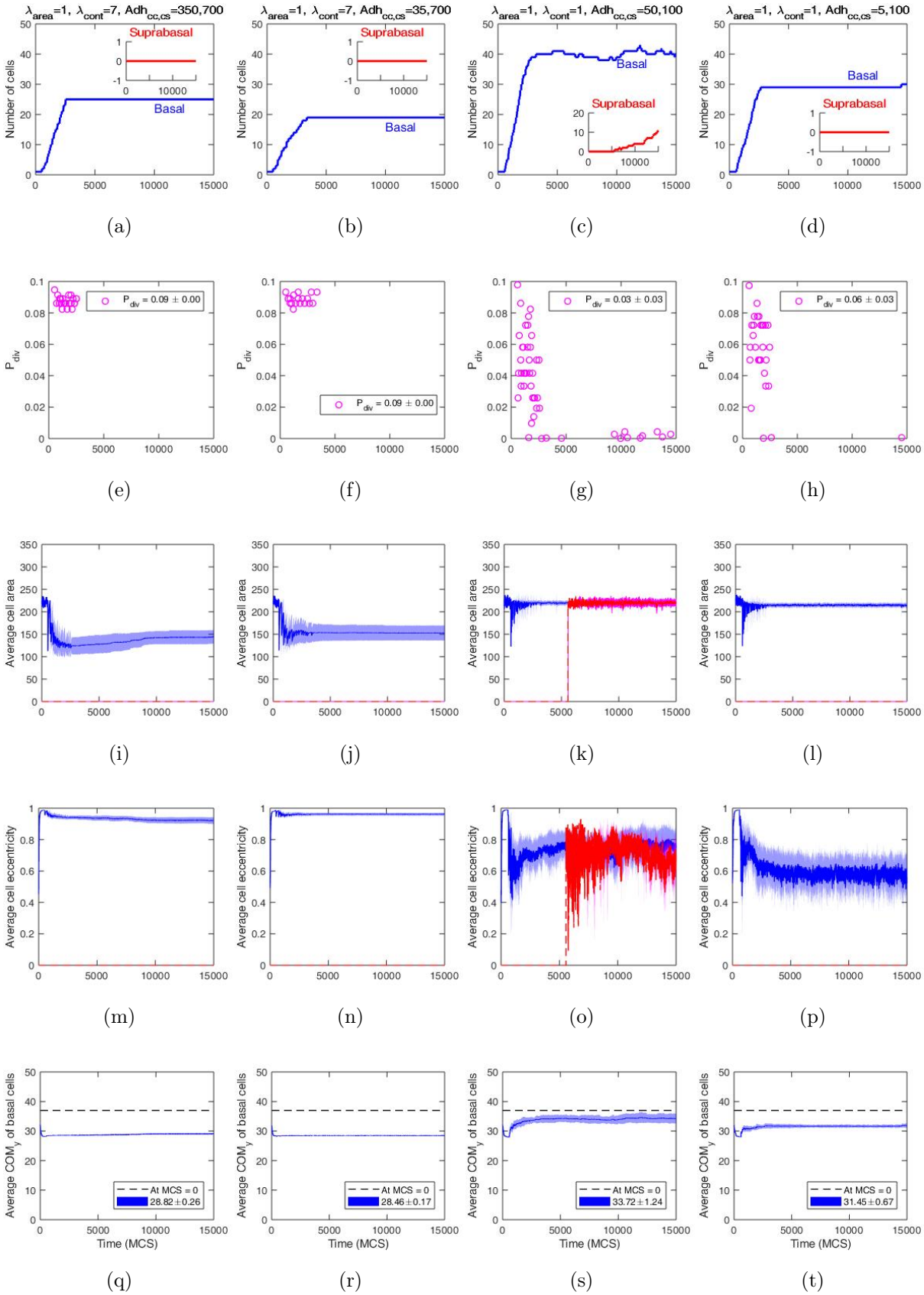


Figure 16: Analysis of different configurations of cells at low area strength ($\lambda_{\text{area}} = 1$), where cell division orientation is perpendicular to the substrate.

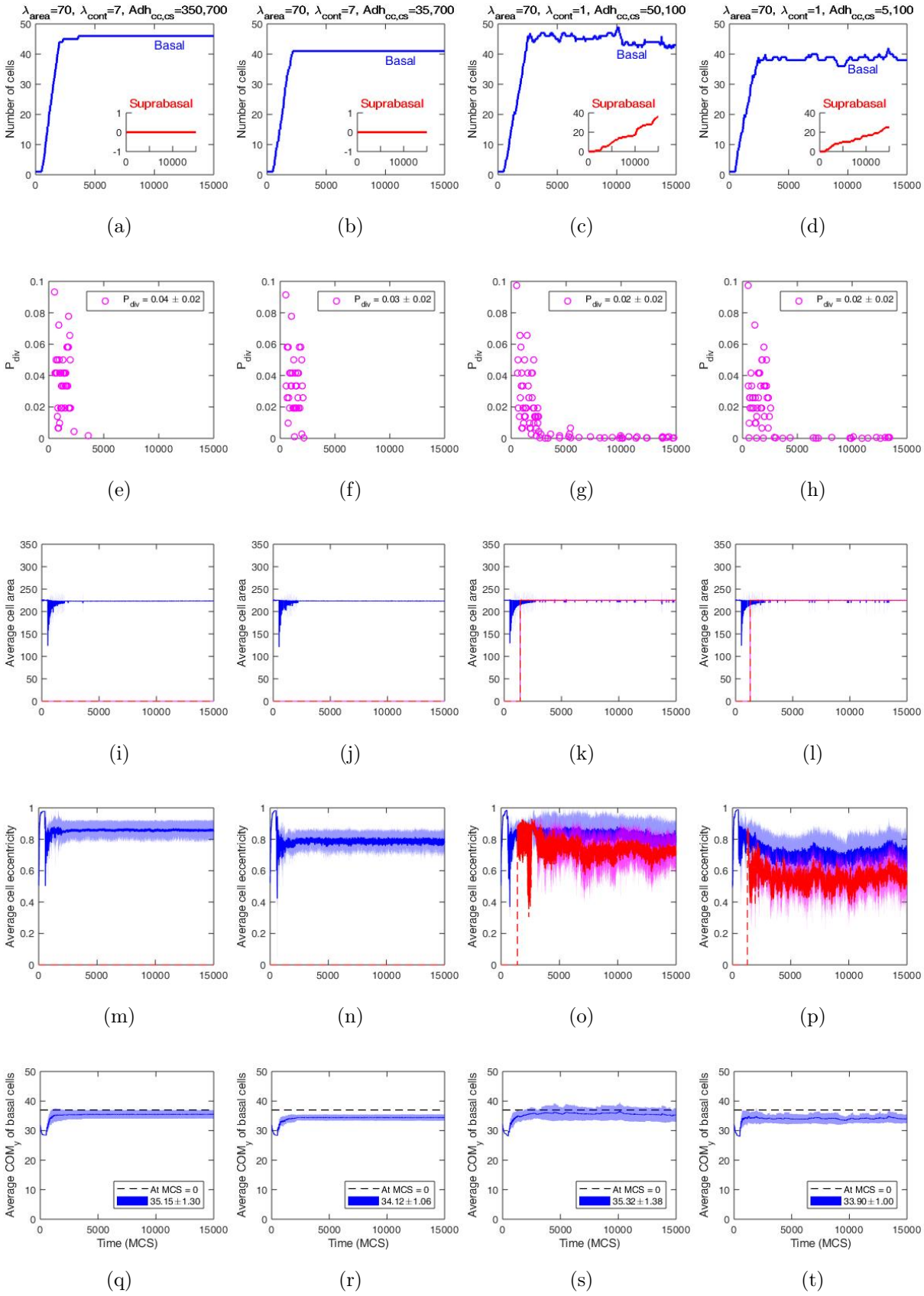


Figure 17: Analysis of different configurations of cells at high area strength ($\lambda_{\text{area}} = 70$), where cell division orientation is perpendicular to the substrate.

To assess the consistency of the multicellular formations, a number of simplified numerical models were proposed, based on the number of cells attached to the substrate. Similar to in single cell morphology, energy was estimated and hence minimised to form solutions for individual cell length and height, and this was subsequently compared via cell size and cell aspect ratios to the simulated results.

These methods can be viewed in Appendix B. There was quite a significant error margin on these when estimated solutions were compared against the CC3D numerical output. There is therefore significant potential for future works to take a more complex, refined analytical approach to this numerical estimation for multicellular regimes.

5 Conclusion

In this project, we introduced a minimal computational model to investigate the emergence of collective morphology of epithelial cells. We used the Cellular Potts Model to form the basis of our theory. This model allowed us to simulate diverse collective morphology using various combinations of mechanical properties of cells and the orientation of cell division axis in the software package CompuCell3D.

First, we simulated the morphology of a single cell for various internal cell mechanisms and adhesion parameters. We found that the squamous (i.e., flat)-to-cuboidal shape transition is promoted by increasing cell contractility. A similar shape transition is also found with decreasing cell-substrate adhesion. A numerical model based on an individual rectangular cell was used to assess the consistency of these simulation results. We predicted that increasing cell-substrate adhesion and weakening cell cortex contractility encourages the formation of mono- and multilayered structures, due to increased proliferation probability of individual cells. The cell size coefficient was found to increase the number of cells for any contractility and have the dominant effect on proliferation probability.

Following this, cell proliferation was incorporated into the model, where the strength of cell compressibility has the predominant effect on cell growth, and thus proliferation. Monolayers of flat, cuboidal and tall cells were found when the axis of cell proliferation is perpendicular to the substrate, mirroring biologically observed epithelial formations. Here, the proliferation of a single cell type on a substrate was demonstrated, achieving our main project aim. The monolayer to multilayer transition was promoted via cell extrusion, dependent on the mechanical properties of cells and the orientation of cell division. Both monolayered and multilayered simulation results were found to be consistent with experimental observations.

Overall, this project proposed a computational model that maps out phase diagrams of multicellular morphologies as functions of cellular properties and the orientation of cell division. Since the phase diagrams characterise cellular properties and mechanics that affect tissue geometry, they can provide new insights for drug development and targeted therapies for disease associated with morphologies of epithelial tissues.

5.1 Future Research Recommendations

Overall, we have achieved the project aim of demonstrating the proliferation of an epithelial structure from a single cell on a substrate, and identified the effects of the internal mechanisms under which cells operate in multicellular tissues, consistent with experimental observations.

A significant recommendation for future work would be to build on our model by introducing cell death to model the self-regulation experimentally observed of multilayered epithelial regimes. An additional recommendation would be to introduce a few cells whose properties differ from the majority of the cells, due to mutations (e.g., cancer cells). An analysis of cellular interactions and morphologies with various cell types can provide new results on the role of mutated cells in regulting the tissue morphology. Finally, one can develop a refined theoretical investigation

into monolayered structures to assess the consistency of our simulation findings. Appendix B demonstrates a number of numerical models we assessed preliminary. These preliminary models can be analysed in further details.

Acknowledgements

I would like to thank Dr Hamid Khataee for his immense support, guidance and knowledge that proved an endless amount of help during this project.

References

[1] S. van Helvert, S. Cornelis, P. Friedl, Mechanoreciprocity in cell migration, Nature Cell Biology 20 (1) (2018) p.8–20. doi:10.1038/s41556-017-0012-0.

[2] B. Ladoux, R. M. Mège, Mechanobiology of collective cell behaviours, Nature Reviews: Molecular Cell Biology 18 (12) (2017) p.743–75. doi:10.1038/nrm.2017.98.

[3] M. Deforet, V. Hakim, H. Yevick, G. Duclos, P. Silberzan, Emergence of collective modes and tri-dimensional structures from epithelial confinement, Nature Communications 5 (1) (2014) 3747. doi:10.1038/ncomms4747.

[4] S. R. K. Vedula, M. C. Leong, T. L. Lai, P. Hersen, A. J. Kabla, C. T. Lim, B. Ladoux, Emerging modes of collective cell migration induced by geometrical constraints, Proceedings of the National Academy of Sciences - PNAS 109 (32) (2012) p.12974–12979. doi:10.1073/pnas.1119313109.

[5] S. M. Yu, B. Li, S. Granick, Y.-K. Cho, Mechanical Adaptations of Epithelial Cells on Various Protruded Convex Geometries, Cells 9 (6) (2020) p.1434. doi:10.3390/cells9061434.

[6] G. Charras, A. Yap, Tensile Forces and Mechanotransduction at Cell–Cell Junctions, Current Biology 28 (8) (2018) p.R445–R457. doi:10.1016/j.cub.2018.02.003.

[7] D. J. Montell, Morphogenetic Cell Movements: Diversity from Modular Mechanical Properties, Science 322 (5907) (2008) 1502–1505. doi:10.1126/science.1164073.

[8] V. Kostiou, M. W. Hall, P. H. Jones, B. A. Hall, Different responses to cell crowding determine the clonal fitness of p53 and notch inhibiting mutations in squamous epithelia, MRC Cancer Unit, University of Cambridge (2020). doi:10.1101/2020.03.30.015917.

[9] H. Khataee, A. Czirok, Z. Neufeld, Multiscale modelling of motility wave propagation in cell migration, Nature: Scientific Reports 10 (1) (2020) p.8128. doi:10.1038/s41598-020-63506-6.

[10] S. R. K. Vedula, G. Peyret, I. Cheddadi, t. Chen, A. Brugués, H. Hirata, H. Lopez-Menendez, Y. Toyama, L. Neves de Almeida, X. Trepat, C. T. Lim, B. Ladoux, Mechanics of epithelial closure over non-adherent environments, Nature Communications 6 (1) (2015) p.6111. doi:10.1038/ncomms7111.

[11] R. Vincent, E. Bazellières, C. Pérez-González, M. Uroz, X. Serra-Picamal, X. Trepat, Active Tensile Modulus of an Epithelial Monolayer, Physical Review Letters 115 (24) (2015) 248103. doi:10.1103/PhysRevLett.115.248103.

[12] S. F. Pedersen, E. K. Hoffmann, I. Novak, Cell volume regulation in epithelial physiology and cancer, Frontiers in Physiology 4 (2013). doi:10.3389/fphys.2013.00233.

[13] J. G. Betts, K. Young, J. Wise, E. Johnson, B. Poe, D. Kruse, O. Korol, J. Johnson, M. Womble, P. DeSaix, Anatomy and Physiology, OpenStax, Houston, Texas, 2013.
URL https://openstax.org/books/anatomy-and-physiology/pages/4-2-epithelial-tissue

[14] M. A. Schwartz, R. K. Assoian, Integrins and cell proliferation: regulation of cyclindependent kinases via cytoplasmic signaling pathways, Journal of Cell Science 114 (14) (2001) 2553–2560. doi:10.1126/science.276.5317.1425.

[15] C. Brakebusch, D. Bouvard, F. Stanchi, T. Sakai, R. Fässler, Integrins in invasive growth, Journal of Clinical Investigation 109 (8) (2002) 999–1006. doi:10.1172/JCI0215468.

[16] Wikipedia contributors, Cell proliferation — Wikipedia, the free encyclopedia, [Online; accessed 1-October-2021] (2021).
URL https://en.wikipedia.org/w/index.php?title=Cell_proliferation&oldid=1040390021

[17] T. M. Finegan, D. T. Bergstralh, Division orientation: disentangling shape and mechanical forces, Cell Cycle 18 (11) (2019) 1187–1198. doi:10.1080/15384101.2019.1617006.

[18] C. Collinet, T. Lecuit, Stability and dynamics of cell-cell junctions, Progress in Molecular Biology and Translational Science 116 (9) (2013) 25–47. doi:https://doi.org/10.1016/B978-0-12-394311-8.00002-9.

[19] S. A. Hacking, A. Khademhosseini, Cells and surfaces in vitro, in: B. D. Ratner, A. S. Hoffman, F. J. Schoen, J. E. Lemons (Eds.), Biomaterials Science An Introduction to Materials in Medicine, Elsevier, 2013, pp. 408–427.

[20] P. P. Provenzano, P. J. Keely, Mechanical signaling through the cytoskeleton regulates cell proliferation by coordinated focal adhesion and Rho GTPase signaling, Journal of Cell Science 124 (8) (2011) 1195–1205. doi:10.1242/jcs.067009.

[21] A. Mohan, K. T. Schlue, A. F. Kniffin, C. R. Mayer, A. A. Duke, V. Narayanan, P. T. Arsenovic, K. Bathula, B. E. Danielsson, S. P. Dumbali, V. Maruthamuthu, D. E. Conway, Spatial Proliferation of Epithelial Cells

Is Regulated by E-Cadherin Force, Biophysical Journal 115 (5) (2018) 853–864. doi:10.1016/j.bpj.2018.07.030.

[22] C. S. Chen, Geometric Control of Cell Life and Death, Science 276 (5317) (1997) 1425–1428. doi:10.1126/science.276.5317.1425.

[23] D. P. Doupé, A. M. Klein, B. D. Simons, P. H. Jones, The ordered architecture of murine ear epidermis is maintained by progenitor cells with random fate, Developmental Cell 18 (2) (2010) 317–323. doi:10.1016/j.devcel.2009.12.016.

[24] S. J. Streichan, C. R. Hoerner, T. Schneidt, D. Holzer, L. Hufnagel, Spatial constraints control cell proliferation in tissues, Proceedings of the National Academy of Sciences 111 (15) (2014) 5586–5591. doi:10.1073/pnas.1323016111.

[25] A. Tzur, R. Kafri, V. S. LeBleu, G. Lahav, M. W. Kirschner, Cell Growth and Size Homeostasis in Proliferating Animal Cells, Science 325 (5937) (2009) 167–171. doi:10.1126/science.1174294.

[26] A. Puliafito, L. Hufnagel, P. Neveu, S. Streichan, A. Sigal, D. K. Fygenson, B. I. Shraiman, Collective and single cell behavior in epithelial contact inhibition, Proceedings of the National Academy of Sciences 109 (3) (2012) 739–744. doi:10.1073/pnas.1007809109.

[27] R. Farhadifar, J.-C. Röper, B. Aigouy, S. Eaton, F. Jülicher, The Influence of Cell Mechanics, Cell-Cell Interactions, and Proliferation on Epithelial Packing, Current Biology 17 (24) (2007) 2095–2104. doi:10.1016/j.cub.2007.11.049.

[28] P. Niewiadomska, D. Godt, U. Tepass, De-cadherin is required for intercellular motility during drosophila oogenesis, Journal of Cell Biology (JCB) 144 (3) (1999) 533–547. doi:10.1083/jcb.144.3.533.

[29] C. Medioni, S. Noselli, Dynamics of the basement membrane in invasive epithelial clusters in *Drosophila*, Development 132 (13) (2005) 3069–3077. doi:10.1242/dev.01886.

[30] M. Melani, K. J. Simpson, J. S. Brugge, D. Montell, Regulation of Cell Adhesion and Collective Cell Migration by Hindsight and Its Human Homolog RREB1, Current Biology 18 (7) (2008) 532–537. doi:10.1016/j.cub.2008.03.024.

[31] J. M. Gomez, Y. Wang, V. Riechmann, Tao controls epithelial morphogenesis by promoting Fasciclin 2 endocytosis, Journal of Cell Biology 199 (7) (2012) 1131–1143. doi:10.1083/jcb.201207150.

[32] C. Berte, L. Sulak, T. Lecuit, Myosin-dependent junction remodelling controls planar cell intercalation and axis elongation, Nature 429 (2004) 667–671.

[33] T. Otain, T. Ichii, S. Aono, M. Takeichi, Cdc42 gef tuba regulates the junctional configuration of simple epithelial cells, J. Cell Biol 175 (2006) 135–146.

[34] J. D. Franke, R. A. Montague, D. P. Kiehart, Non- muscle myosin ii generates forces that transmit tension and drive contraction in multiple tissues during dorsal closure, Curr. Biol 15 (2005) 2208–2221.

[35] E. Paluch, After the Greeting: Realizing the Potential of Physical Models in Cell Biology, Trends in Cell Biology 25 (12) (2015) p.711–713. doi:10.1016/j.tcb.2015.08.001.

[36] J. Obsbourne, A. Fletcher, J. Pitt-Francis, P. Maini, D. Gavaghan, Comparing individual-based approaches to modelling the self-organization of multicellular tissues, PLOS Computational Biology 13 (2) (2017) p.e1005387–e1005387. doi:10.1371/journal.pcbi.1005387.

[37] J. M. Nava-Sedeño, A. Voß-Böhme, H. Hatzikirou, A. Deutsch, F. Peruani, Modelling collective cell motion: are on- and off-lattice models equivalent?, Philosophical Transactions of the Royal Society B 375 (1807) (2002) 999–1006. doi:10.1098/rstb.2019.0378.

[38] F. Graner, J. Glazier, Simulation of biological cell sorting using a two-dimensional extended potts model, Physics Review Lettts (PRL) 69 (13) (1992) 2013–7. doi:10.1103/PhysRevLett.69.2013.

[39] J. A. Glazier, F. Graner, Simulation of the differential adhesion driven rearrangement of biological cells, Physical Review E 47 (3) (1993) 2128–2154. doi:10.1103/PhysRevE.47.2128.

[40] N. J. Savill, P. Hogeweg, Modelling morphogenesis: From single cells to crawling slugs, Journal of Theoretical Biology 184 (3) (1997) 229–325. doi:10.1006/jtbi.1996.0237.

[41] R. Giniūnaitė, R. E. Baker, P. M. Kulesa, P. K. Maini, Modelling collective cell migration: neural crest as a model paradigm, Journal of Mathematical Biology (2019). doi:10.1007/s00285-019-01436-2.

[42] A. R. Noppe, A. P. Roberts, A. S. Yap, G. A. Gomez, Z. Neufeld, Modelling wound closure in an epithelial cell sheet using the cellular potts model, Integrative Biology 7 (10) (2015) 1253–1264. doi:10.1039/C5IB00053J.

[43] P. J. Albert, U. S. Schwarz, Dynamics of cell ensembles on adhesive micropatterns: Bridging the gap between single cell spreading and collective cell migration, PLOS Computational Biology 12 (4) (2016) e1004863. doi:10.1371/journal.pcbi.1004863.

[44] F. Thüroff, A. Goychuk, M. Reiter, E. Frey, Bridging the gap between single-cell migration and collective dynamics, eLife 8 (2019) e46842. doi:10.7554/eLife.46842.

[45] M. Swat, G. Thomas, J. Belmonte, A. Shirinifard, D. Hmeljack, J. Glazier, Multi-Scale Modeling of Tissues Using CompuCell3D, Methods in Cell Biology 110 (2020) p.325–366. doi:10.1016/B978-0-12-388403-9.00013-8.

[46] M. Santillán, On the Use of the Hill Functions in Mathematical Models of Gene Regulatory Networks, Mathematical Modelling of Natural Phenomena 3 (2) (2008) 85–97. doi:10.1051/mmnp:2008056.

[47] T. J. Widmann, C. Dahmann, Dpp signaling promotes the cuboidal-to-columnar shape transition of Drosophila wing disc epithelia by regulating Rho1, Journal of Cell Science 122 (9) (2009) 1362–1373. doi:10.1242/jcs.044271.

[48] A. Garcia, A. Vega, D. Boettiger, Modulation of Cell Proliferation and Differentiation through Substrate-dependent Changes in Fibronectin Conformation, Molecular Biology of the Cell 10 (3) (1999) 785–798. doi:10.1091/mbc.10.3.785.

[49] E. Rognoni, F. M. Watt, Skin Cell Heterogeneity in Development, Wound Healing, and Cancer, Trends in Cell Biology 28 (9) (2018) 709–722. doi:10.1016/j.tcb.2018.05.002.

[50] D. Haensel, S. Jin, P. Sun, R. Cinco, M. Dragan, Q. Nguyen, Z. Cang, Y. Gong, R. Vu, A. L. MacLean, K. Kessenbrock, E. Gratton, Q. Nie, X. Dai, Defining Epidermal Basal Cell States during Skin Homeostasis and Wound Healing Using Single-Cell Transcriptomics, Cell Reports 30 (11) (2020) 3932–3947.e6. doi:10.1016/j.celrep.2020.02.091.

[51] K. L. Weber, R. S. Fischer, V. M. Fowler, Tmod3 regulates polarized epithelial cell morphology, Journal of Cell Science 120 (20) (2007) 3625–3632. doi:10.1242/jcs.011445.

[52] E. Hannezo, J. Prost, J.-F. Joanny, Theory of epithelial sheet morphology in three dimensions, Proceedings of the National Academy of Sciences 111 (1) (2014) 27–32. doi:10.1073/pnas.1312076111.

[53] K. Dasbiswas, E. Hannezo, N. S. Gov, Theory of Epithelial Cell Shape Transitions Induced by Mechanoactive Chemical Gradients, Biophysical Journal 114 (4) (2018) 968–977. doi:10.1016/j.bpj.2017.12.022.

A Additional Single Cell Numerical Morphology

To assess the validity of our single cell morphology simulation results, we estimate the phenomological Potts energy function for an additional cell configuration. Here, we assume a single cell with an ovular shape, with base length l and height w. This gives Equation 1 in the form:

$$E(l,h;\lambda_{\text{area}},\lambda_{\text{cont}},\lambda_{\text{adh}_{\text{cs}}}) = \lambda_{\text{area}}\left(\frac{1}{4}\pi lw - A_0\right)^2 + \lambda_{\text{cont}}\left(\frac{\pi}{\sqrt{2}}\sqrt{l^2+w^2}\right)^2 - \lambda_{\text{adh}_{\text{cs}}}l, \tag{7}$$

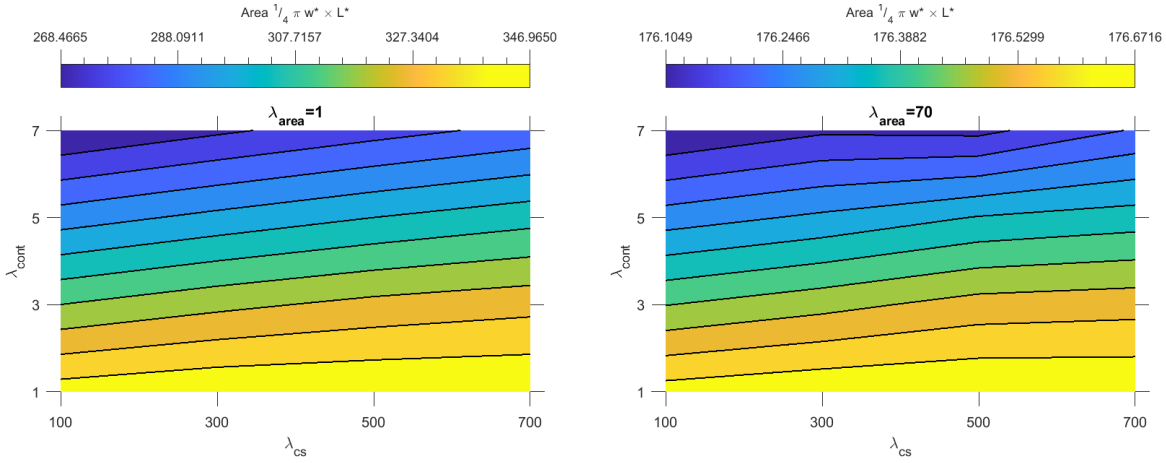
To determine the minima of this energy, we take partial derivatives as follows:

$$\begin{aligned}\frac{\partial E(l,w)}{\partial l} = 0 \longrightarrow \frac{1}{2}\lambda_{\text{area}}\pi w\left(\frac{1}{4}\pi lw - A_0\right) + \lambda_{\text{cont}}\pi^2 l - \lambda_{\text{adh}_{\text{cs}}} = 0.\\ \frac{\partial E(l,h)}{\partial l} = 0 \longrightarrow \frac{1}{2}\lambda_{\text{area}}\pi l\left(\frac{1}{4}\pi lh - A_0\right) + \lambda_{\text{cont}}\pi^2 h = 0.\end{aligned} \tag{8}$$

Solving 8 for l and w gives:

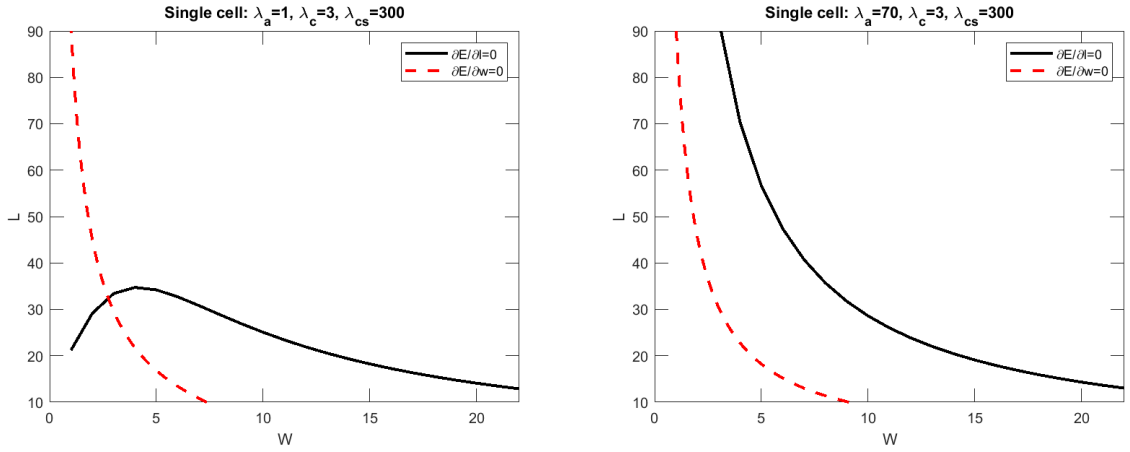
$$\begin{aligned} l &= \frac{4\left(\lambda_{\text{area}}\pi hA_0 + 2\lambda_{\text{adh}_{\text{cs}}}\right)}{\pi^2\left(\lambda_{\text{area}}l^2 + 8\lambda_{\text{cont}}\right)}\\ w &= \frac{4\lambda_{\text{area}}\pi lA_0}{\pi^2\left(\lambda_{\text{are}}l^2 + 8\lambda_{\text{cont}}\right)}\end{aligned} \tag{9}$$

Evaluating at various λ parameters to compare our single cell morphology simulations gives Fig. 18 below. If we compare this to the single cell morphology phase diagram, Figure 7, we can see that this is generally consistent with our shape transitions. However, when assessing this cell regime against example solutions, shown in Figure 19, we find that it appears that there is no real solution to Equation 8 for high $\lambda_{\text{area}}(=70)$ and only a single solution for low $\lambda_{\text{area}}(=1)$.



(a) (b)

Figure 18: Equilibrium single-cell area, assuming an ellipse with height w and length l, derived from Equations (9).



(a) (b)

Figure 19: Example solutions for Equations (8)

.

B Multicellular Numerical Analysis

To analytically express steady area A^* of cells such that energy E of the system is minimised, we consider configurations of cells in Fig. 20 which allow to write energy E as a function of length l and width w of the cells. The minimum of energy is driven by setting:

$$\frac{\partial E(l,w)}{\partial l} = 0, \quad \frac{\partial E(l,w)}{\partial w} = 0 \tag{10}$$

which results in length l^* and width w^* at mechanical equilibrium, and thus the corresponding steady cell area $A^*(l^*, w^*)$. This steady cell area A^* can then explicitly determine proliferation condition, i.e., $A* > A_0$.

Figure 20: Analytical representation of a system of multiple cells.

Potts energy E for a fixed number of N rectangular cells is written:

$$E(l,w) = \Big(\lambda_{\text{area}}\,(lw - A_0)^2 + \lambda_{\text{cont}}(2l + 2w)^2 - \lambda_{\text{adh}_{\text{cs}}} l - \lambda_{\text{adh}_{\text{cc}}} 2w\Big) N \tag{11}$$

where N is large $\gg 1$ and the boundary cells do not affect the calculations.

The minimum of this energy is determined by:

$$\begin{aligned}
\frac{\partial E(l,w)}{\partial l} = 0 &\longrightarrow \lambda_{\text{area}} 2lw^2 - \lambda_{\text{area}} 2wA_0 + \lambda_{\text{cont}} 8l + \lambda_{\text{cont}} 8w - \lambda_{\text{adh}_{\text{cs}}} = 0. \\
\frac{\partial E(l,w)}{\partial w} = 0 &\longrightarrow \lambda_{\text{area}} l^2 2w - \lambda_{\text{area}} 2lA_0 + \lambda_{\text{cont}} 8l + \lambda_{\text{cont}} 8w - 2\lambda_{\text{adh}_{\text{cc}}} = 0.
\end{aligned} \tag{12}$$

It seems there is no general real solution for Equations (12). Equations (12) are evaluated at various λ parameters correspond to different cell morphologies and solved; see Fig. 30

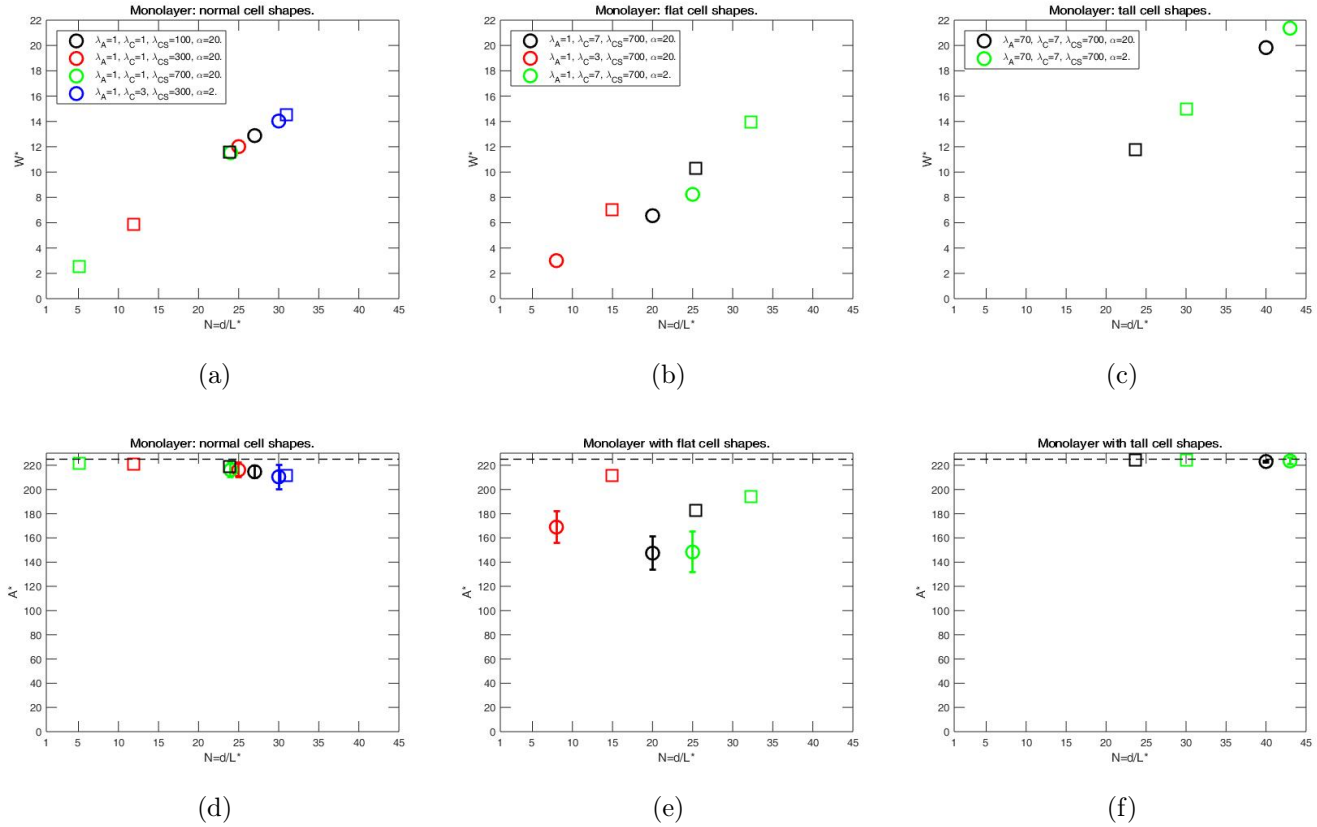


Figure 21: Equilibrium cell height w^* and area A^* versus number of cells N calculated with λ parameters correspond to monolayers of flat, normal, and tall cells. Squares: w^* and $A^* = w^* l^*$ obtained by solving Equations (12) versus $N = d/l^*$. Circles A^*: calculated by CC3D versus the steady number of cells N in simulations. These CC3D calculations result in $w^* = A^*/l^*$ (circles), where $l^* = d/N$.

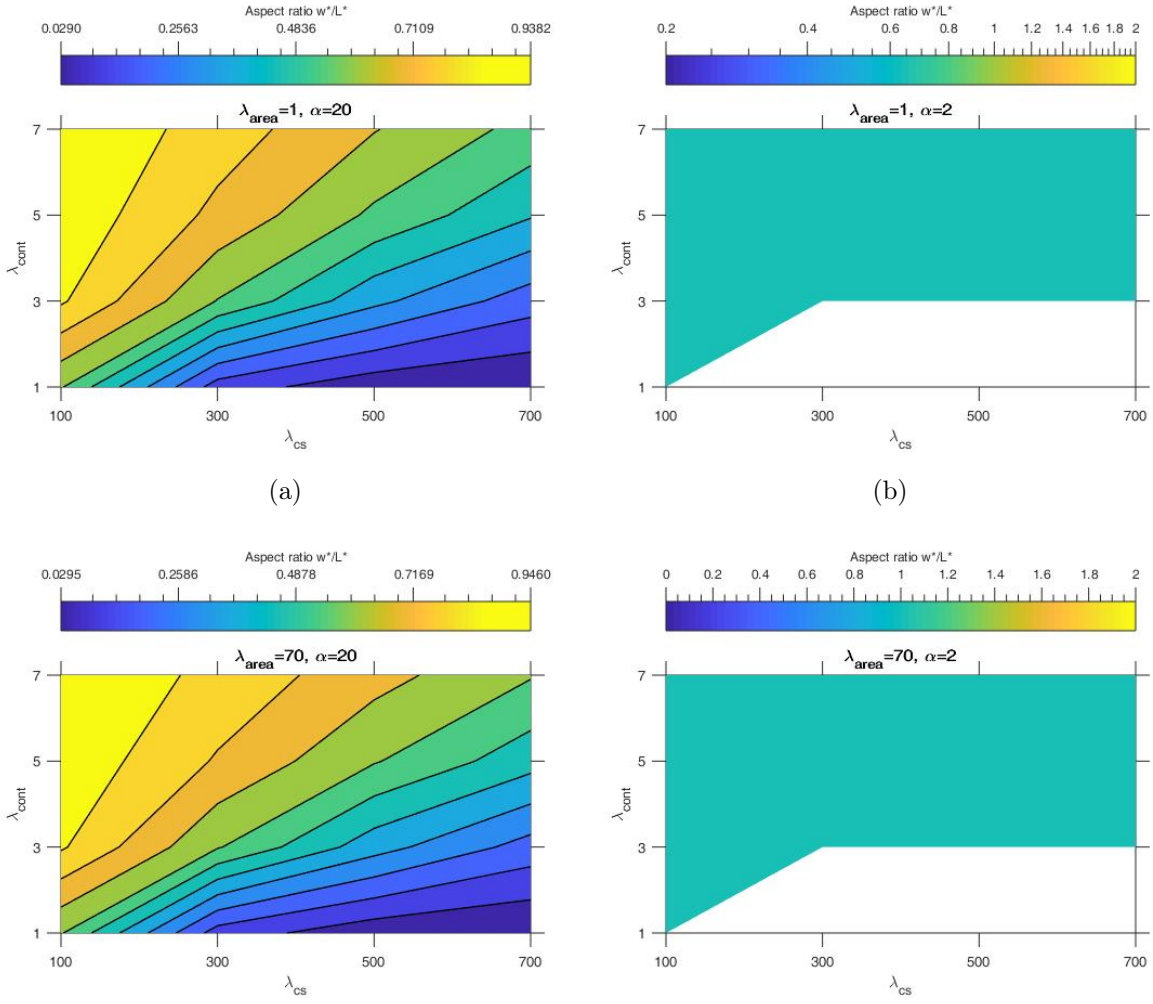


Figure 22: Equilibrium cell aspect ratio w^*/L^* derived by solving Equations (12). (b, d) The blank white part corresponds to parameters where there are three real solutions. In the remaining part in (b, d), $w^*/L^* = 1$

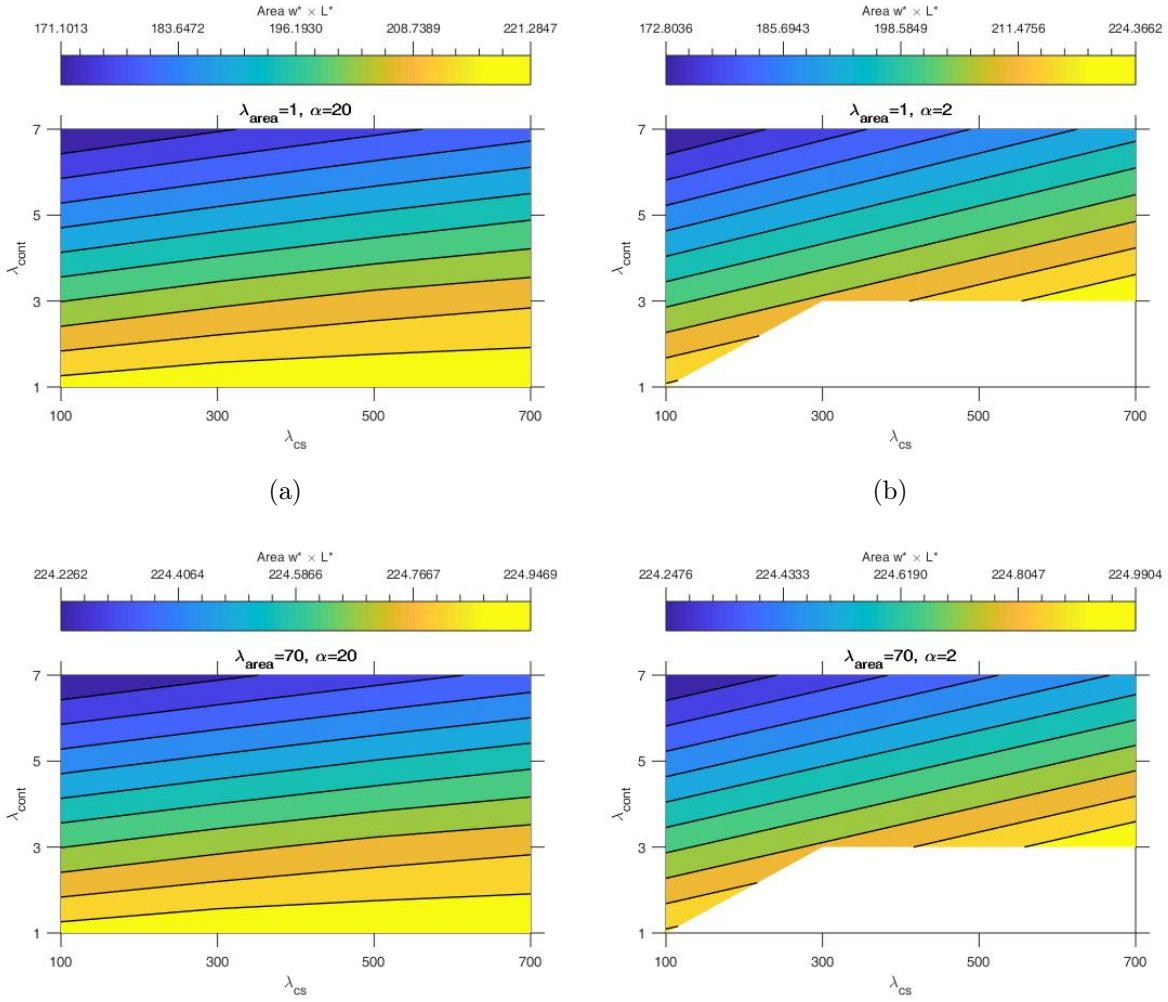


Figure 23: Equilibrium cell area $w^* \times L^*$ derived by solving Equations (12). (b, d) The blank white part corresponds to parameters where there are three real solutions

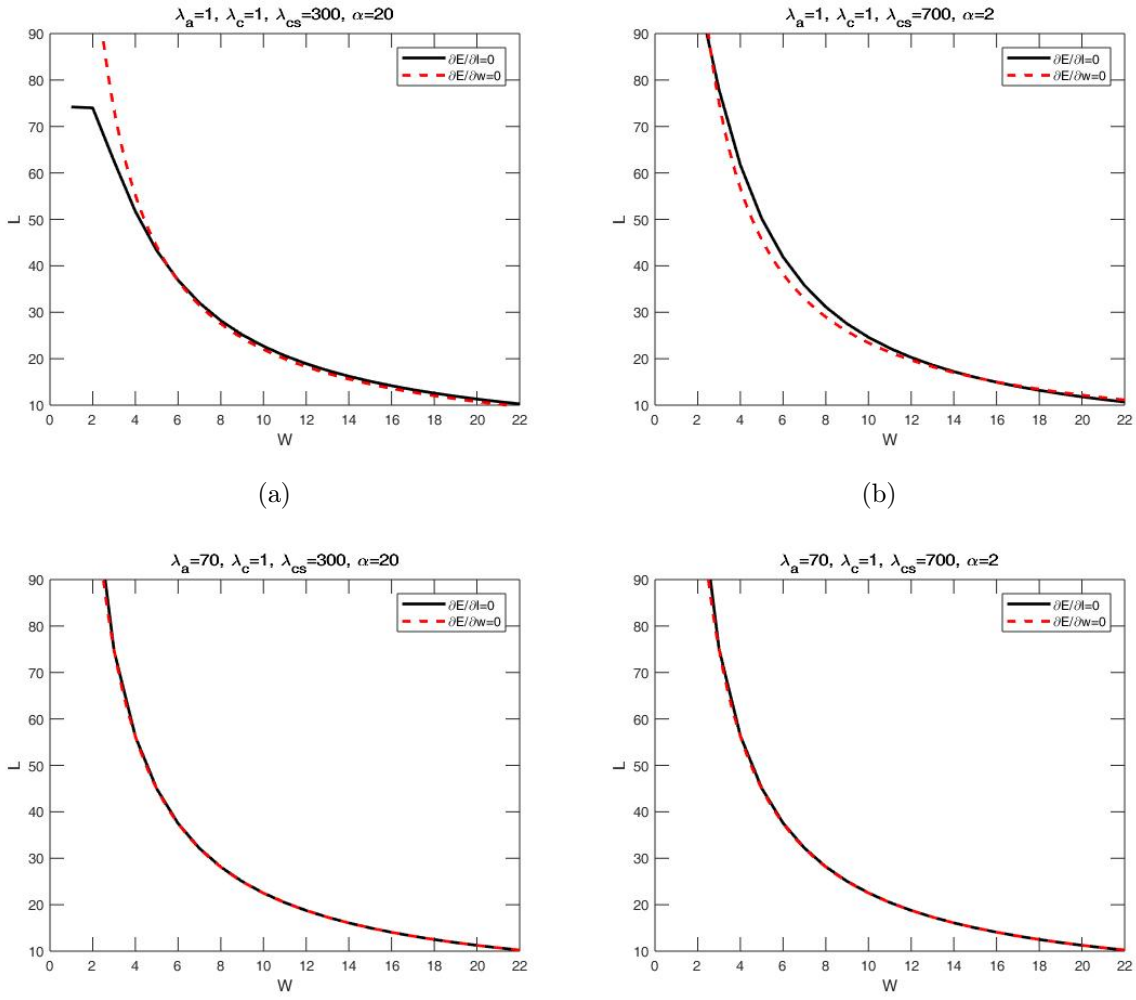


Figure 24: Example solutions for Equations (12). (a) For normal cell shapes $L^* = 37.82, W^* = 5.84$. (b) For thin shapes, there are three solutions $(L^* = 84.85, W^* = 2.65), (L^* = 2.65, W^* = 84.85)$, and $(L^* = 15.48, W^* = 15.48)$. (c) At high $\lambda_{\text{area}} = 70$, Mathematica returns one positive real solution $L^* = 37.84, W^* = 5.94$. But, it seems (visually) there are more solutions. (d) At high $\lambda_{\text{area}} = 70$ and this cell shapes, three real solutions are found $(L^* = 84.85, W^* = 2.65), (L^* = 2.65, W^* = 84.85)$, and $(L^* = 15.01, W^* = 15.01)$. Again, at high $\lambda_{\text{area}} = 70$ it seems (visually) there are more solutions.

Second approach: Assume that at mechanical equilibrium N cells (each $l \times w$) are configured on the substrate with a length of d (Fig. 25).

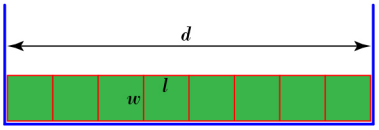


Figure 25: Analytical representation of a system of multiple cells.

Form Eq. (11), the energy E of the system reads,

$$\begin{aligned} E(l,w) &= \Big(\lambda_{\text{area}}\left(lw - A_0\right)^2 + \lambda_{\text{cont}}(2l+2w)^2 - \lambda_{\text{adh}_{\text{cs}}} l - \lambda_{\text{adh}_{\text{cc}}} 2w\Big)N. \\ E(w) &= \Big(\lambda_{\text{area}}\left(\frac{d}{N}w - A_0\right)^2 + \lambda_{\text{cont}}(2\frac{d}{N}+2w)^2 - \lambda_{\text{adh}_{\text{cs}}}\frac{d}{N} - \lambda_{\text{adh}_{\text{cc}}} 2w\Big)N, \quad \text{where } l = \frac{d}{N}. \end{aligned} \tag{13}$$

The minimum of this energy is determined by:

$$\frac{dE}{dw} = 0 \rightarrow \frac{2d\lambda_{\text{area}}\left(dw - NA_0\right) + 8N\lambda_{\text{cont}}(d+wN) - 2N^2\lambda_{\text{adh}_{\text{cc}}}}{N} = 0. \tag{14}$$

Solving Eq. 14 results in:

$$w^* = \frac{N\left(dA_0\lambda_{\text{area}} - 4d\lambda_{\text{cont}} + N\lambda_{\text{adh}_{\text{cc}}}\right)}{d^2\lambda_{\text{area}} + 4N^2\lambda_{\text{cont}}}, \text{ and with } l^* = \frac{d}{N},\ A^* = \frac{d\left(dA_0\lambda_{\text{area}} - 4d\lambda_{\text{cont}} + N\lambda_{\text{adh}_{\text{cc}}}\right)}{d^2\lambda_{\text{area}} + 4N^2\lambda_{\text{cont}}}. \tag{15}$$

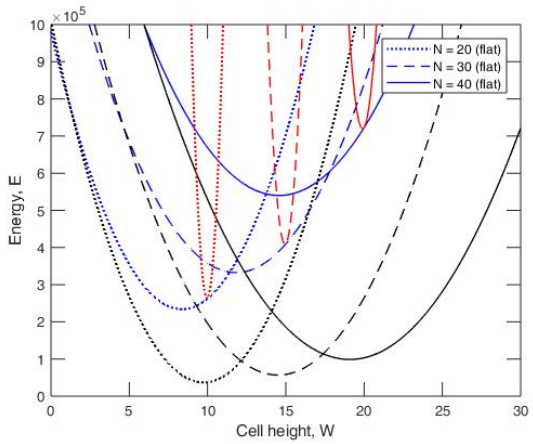


Figure 26: Energy E versus cell height w for monolayers with cell shapes flat (blue), normal (black), and tall (red) at different cell number $N = 20$ (dotted), 30 (dashed), and 40 (solid). Parameters for monolayer of flat cells are $\lambda_{\text{area}} = 1$, $\lambda_{\text{cont}} = 7$, $\lambda_{\text{adh}_{\text{cs}}} = 700$, and $\lambda_{\text{adh}_{\text{cc}}} = 35$. With flat cell shapes, number of cells at mechanical equilibrium remains 20. Parameters for monolayer with normal cell shapes are $\lambda_{\text{area}} = 1$, $\lambda_{\text{cont}} = 1$, $\lambda_{\text{adh}_{\text{cs}}} = 100$, and $\lambda_{\text{adh}_{\text{cc}}} = 5$. Here, number of cells at mechanical equilibrium is 27 (when hitting walls) to 30 (end of simulation). Parameters for monolayer with with tall cells are $\lambda_{\text{area}} = 70$, $\lambda_{\text{cont}} = 7$, $\lambda_{\text{adh}_{\text{cs}}} = 700$, and $\lambda_{\text{adh}_{\text{cc}}} = 35$. At mechanical equilibrium, number of cells reach 40 to 41. E is calculated using Eq. (13).

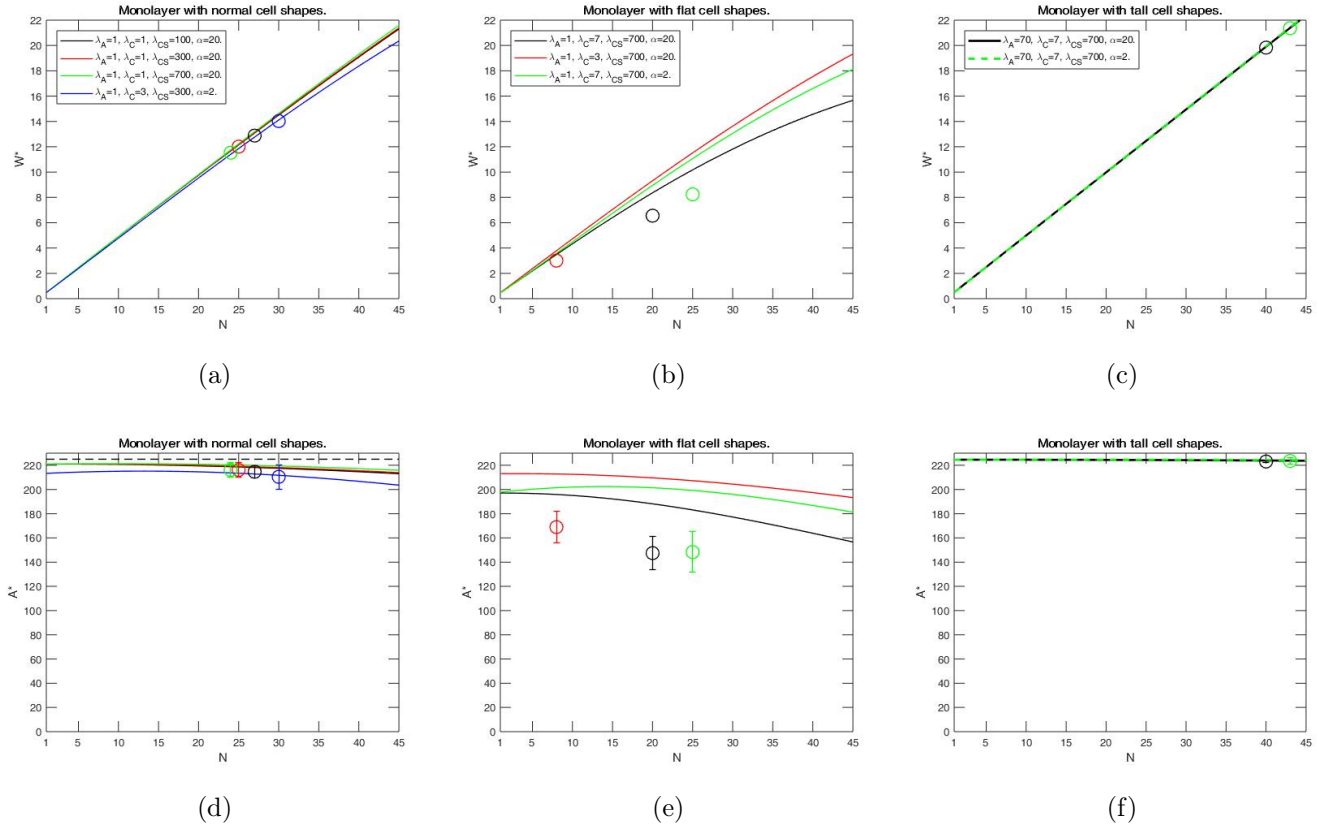
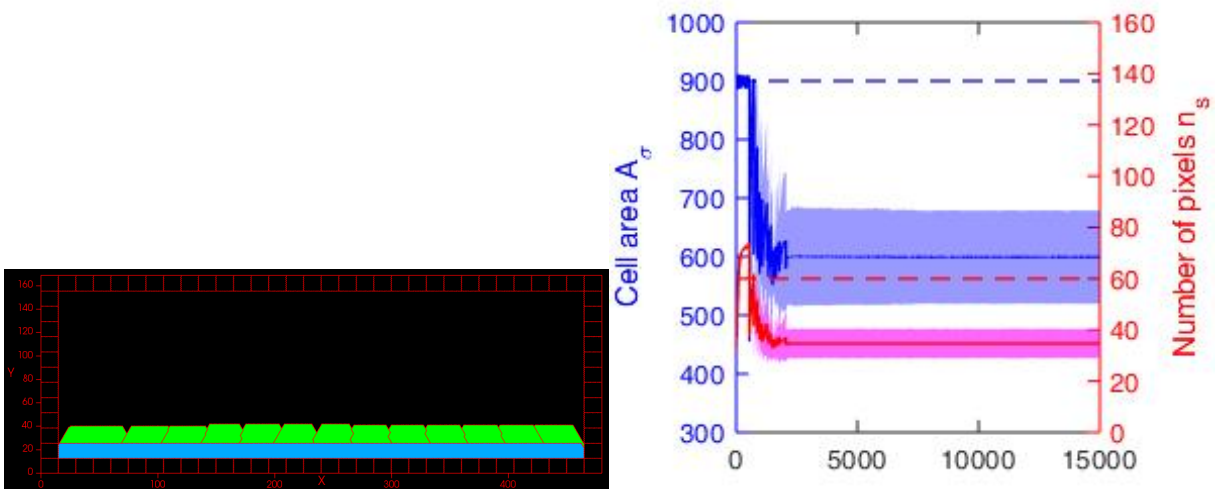


Figure 27: Equilibrium cell height w^* and area A^* versus number of cells N calculated with λ parameters correspond to monolayers of flat, normal, and tall cells. Curves are calculated using Eq. (15). Symbols A^*: calculated by CC3D versus the steady number of cells N in simulations. These CC3D calculations result in $w^* = A^*/l^*$ (symbols), where $l^* = d/N$.



(a) (b)

Figure 28: Simulations for flat cells with larger size (30×30 pixels). Parameters are: $\lambda_{\text{area}} = 1$, $\lambda_{\text{cont}} = 7$, $\lambda_{\text{adh}_{\text{cs}}} = 1400$, and $\lambda_{\text{adh}_{\text{cc}}} = 75$. Here, number of cells at mechanical equilibrium remains 12 to 13 with steady area 615.25 ± 62.18 which gives mean cell height of 16.41. Equation (15) results in $W^* = 22.85$ and $A^* = 856.94$.

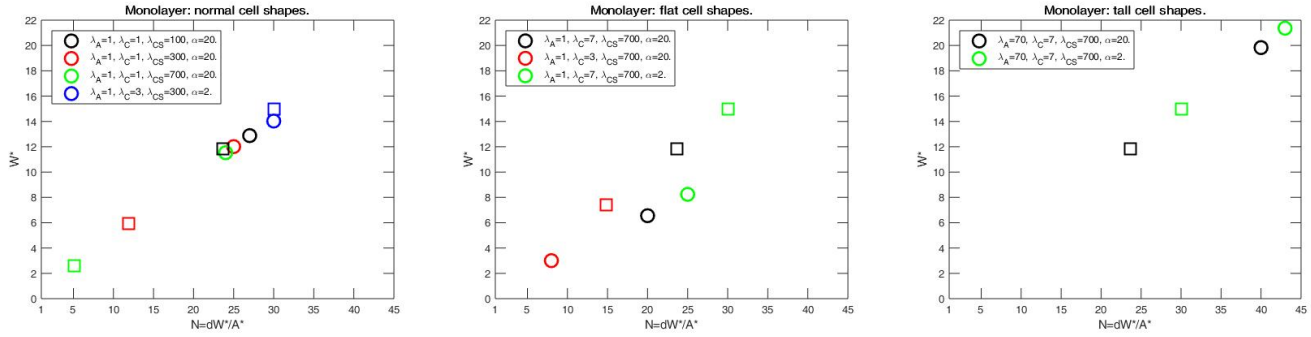
Third approach: Assume that the cell area is equal to the target area (i.e., $A = lw = A_0$) at mechanical equilibrium:

$$E(l,w) = \Big(\lambda_{\text{area}}\,(lw - A_0)^2 + \lambda_{\text{cont}}(2l + 2w)^2 - \lambda_{\text{adh}_{\text{cs}}} l - \lambda_{\text{adh}_{\text{cc}}} 2w\Big) N.$$
$$E(w) = \Big(\lambda_{\text{cont}}(2\frac{A_0}{w} + 2w)^2 - \lambda_{\text{adh}_{\text{cs}}} \frac{A_0}{w} - \lambda_{\text{adh}_{\text{cc}}} 2w\Big) N, \quad \text{given that } l = \frac{A_0}{w}. \quad (16)$$

The minimum of this energy is determined by:

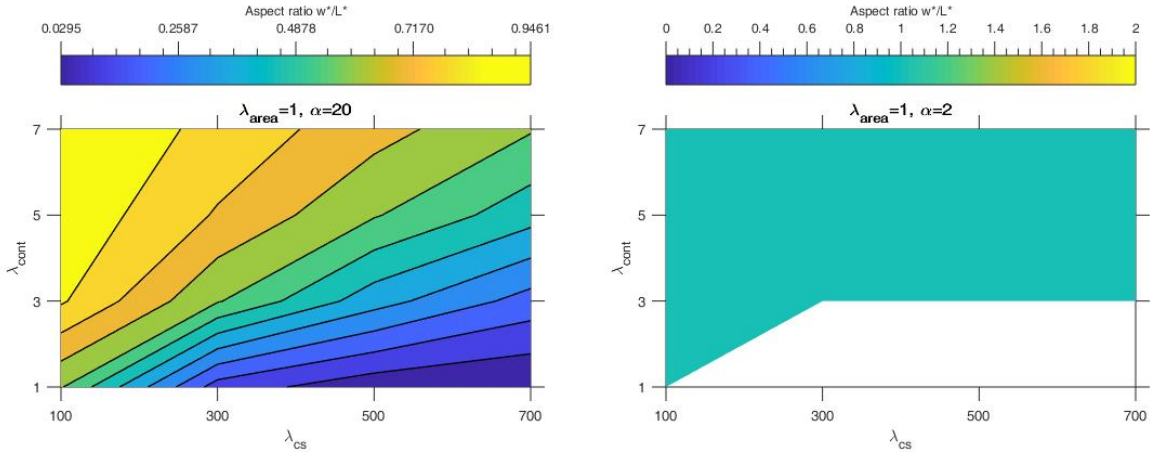
$$\frac{dE}{dw} = 0 \rightarrow \frac{N\left(-\lambda_{\text{cont}} 8A_0^2 + \lambda_{\text{cont}} 8w^4 + \lambda_{\text{adh}_{\text{cs}}} A_0 w - 2w^3 \lambda_{\text{adh}_{\text{cc}}}\right)}{w^3} = 0. \quad (17)$$

Solution to Eq. 17 is a long expression (2 pages in Mathematica). Equation (17) is evaluated at various λ parameters correspond to different cell morphologies and solved; see Fig. 29.



(a) (b) (c)

Figure 29: Equilibrium cell height w^* versus number of cells N calculated with λ parameters correspond to monolayers of flat, normal, and tall cells. Squares: w^* obtained by solving Equation (17) versus $N = d/l^*$, where $l^* = A_0/w^*$. Circles: $w^* = A^*/l^*$ obtained from CC3D simulations versus the steady number of cells N in simulations, where $l^* = d/N$.



(a) (b)

Figure 30: Equilibrium cell aspect ratio w^*/L^* derived by solving Equation (17). Note that increasing $\lambda_{\text{area}} = 70$ will not affect the plots, since Equation (17) is independent of λ_{area}. In (b), the white part in (b) indicates that there are three real solutions $w^* = 7.5, 15$ and 30. In the remain part of (b), $w^*/L^* = 1$, where $W^* = 15$.

C CC3D Code: Steppables (Python)

```
1  ############################################################
2  # Analysis of the collective self-organisation of epithelial layers
       .
3  # Author: Hamid Khataee.
4  # Email: h.khataee@uq.edu.au
5  # 1. Change directory of .txt files to your a directory in your
       machine.
6  # 2. Run.
7  # PLEASE DO NOT REDISTRIBUTE WITHOUT PERMISSION.
8  ############################################################
9  from PySteppables import *
10 import CompuCell
11 import sys
12 #import matplotlib.pyplot as plt
13 import random # Mine
14 #from numpy import *
15 #from math import *
16 import numpy
17 import numpy as np
18 import math
19 import os.path
20 from XMLUtils import dictionaryToMapStrStr as d2mss
21 import random as ra
22 #------------- Global parameters for scanning -------------------
23 Alpha = 1.0
24 targetVol_Normal = 225
25 lambdaVol_Normal = 1
26 targetSurf_Normal = 0
27 lambdaSurface_Normal = 1
28 MCS_Start_Proliferation = 500
29 n_Hill = 10
30 threshold_Hill = float(2.0)
31 threshold_area_div = 1
32 Div_Ori = "Y"
33 # lambdaVol_Mutated = 70
34 # LambdaSurface_Mutated = 3
35 #----------------------------------------------------------------------
36 class Epithelial1Steppable(SteppableBasePy):
37
38     #.........................
39     #.........................
40     #.........................
41     def __init__(self,_simulator,_frequency=1):
42         SteppableBasePy.__init__(self,_simulator,_frequency)
43
44     #.........................
45     #.........................
46     #.........................
47     def start(self):
48         # any code in the start function runs before MCS=0
49         #------- Assign mechanical properties
               ----------------------------
50         for cell in self.cellList:
51             if cell.type == 1: # Normal
52                 cell.targetVolume = targetVol_Normal
53                 cell.lambdaVolume = lambdaVol_Normal
54                 cell.TargetSurface = targetSurf_Normal #set initial
                       boundary length (perimeter) of cells.
55                 cell.lambdaSurface = lambdaSurface_Normal
56             if cell.type == 4: # Membrane
57                 cell.targetVolume = 6750 #Initial volume of the
                       membrane cell = 10*(50*10)=5000 pixels; or
                       15*(30*15)=6750 pixels; or 15*(20*15)=4500.
58                 cell.lambdaVolume = 100
59                 cell.TargetSurface = 0 #set initial boundary length
                       (perimeter) of cells.
60                 cell.lambdaSurface = 100  # A high contractility.
```

```
61          #-------------------------------------------------------------------
62          #................ Assign extra mechanical properties
                ..............
63          for cell in self.cellList:
64              if cell.type == 1:
65  #                cell.dict['velocityX'] = 0
66  #                cell.dict['velocityY'] = 0
67  #                cell.dict['VelocityMagnitude'] = 0
68  #                cell.dict['VelocityAngle'] = 0
69  #                cell.dict['PreviousComX'] = cell.xCOM
70  #                cell.dict['PreviousComY'] = cell.yCOM
71
72                  cell.dict['IsConnectedToSheet'] = 0
73                  cell.dict['IsConnectedToMedium'] = 0
74                  cell.dict['IsConnectedToMembrane'] = 0
75                  cell.dict['NPixelsTotal'] = 0
76                  cell.dict['NPixelsAdjToFree'] = 0
77                  cell.dict['NPixelsAdjToSubstrate'] = 0
78                  cell.dict['NPixelsAdjToFreeSubstrate'] = 0
79  #                cell.dict['RightIsMedium'] = 0
80  #                cell.dict['LeftIsMedium'] = 0
81          #.....................................................................
82          #================= Read XML parameters
                ============================
83          pottsXMLData=self.simulator.getCC3DModuleData("Potts")
84          if pottsXMLData:
85              StepsElement=pottsXMLData.getFirstElement("Steps")
86              if StepsElement:
87                  val_Steps = (StepsElement.getText())
88              temperatureElement=pottsXMLData.getFirstElement("
                    Temperature")
89              if temperatureElement:
90                  val_Temperature = (temperatureElement.getText())
91              Dimension_y=pottsXMLData.getFirstElement("Dimensions",
                    d2mss({"x":"480"}))
92              if Dimension_y:
93                  val_Dimension_y = Dimension_y.getAttributeAsDouble
                        ("y")
94
95          contactXMLData=self.simulator.getCC3DModuleData("Plugin","
                Contact") # get <Plugin Name="Contact"> section of XML
                file
96          if contactXMLData:   # check if we were able to
                successfully get the section from simulator
97              ContactEnergy_NormalNormal = contactXMLData.
                    getFirstElement("Energy",d2mss({"Type1":"Normal","
                    Type2":"Normal"})) # get <Energy Type1="Mutated"
                    Type2="UnMutated"> element
98              if ContactEnergy_NormalNormal: # check if the attempt
                    was succesful
99                  val_ContactEnergy_NormalNormal=(
                        ContactEnergy_NormalNormal.getText()) # get
                        value of <Energy Type1="Mutated" Type2="Mutated
                        "> element and convert it into float
100
101         contactXMLData=self.simulator.getCC3DModuleData("Plugin","
                Contact") # get <Plugin Name="Contact"> section of XML
                file
102         if contactXMLData:   # check if we were able to
                successfully get the section from simulator
103             ContactEnergy_MembraneNormal = contactXMLData.
                    getFirstElement("Energy",d2mss({"Type1":"Membrane","
                    Type2":"Normal"})) # get <Energy Type1="Mutated"
                    Type2="UnMutated"> element
104             if ContactEnergy_MembraneNormal: # check if the attempt
                     was succesful
```

```
105                 val_ContactEnergy_MembraneNormal=(
                        ContactEnergy_MembraneNormal.getText()) # get
                        value of <Energy Type1="Mutated" Type2="Mutated
                        "> element and convert it into float
106
107         contactXMLData=self.simulator.getCC3DModuleData("Plugin","
                Contact") # get <Plugin Name="Contact"> section of XML
                file
108         if contactXMLData:   # check if we were able to
                successfully get the section from simulator
109             ContactEnergy_MediumNormal = contactXMLData.
                    getFirstElement("Energy",d2mss({"Type1":"Medium","
                    Type2":"Normal"})) # get <Energy Type1="Mutated"
                    Type2="UnMutated"> element
110             if ContactEnergy_MediumNormal: # check if the attempt
                    was succesful
111                 val_ContactEnergy_MediumNormal=(
                        ContactEnergy_MediumNormal.getText()) # get
                        value of <Energy Type1="Mutated" Type2="Mutated
                        "> element and convert it into float
112         #=====================================================================

113         #$$$$$$$$$$$$$$$$$$$$$$$$ Print
                $$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
114         File_path = 'C:\Users\s4392898\Desktop\Multi'
115         #File_path = 'E:\Multi'
116         if Alpha == 1:
117             File_path = 'C:\Users\s4392898\Desktop\Multi\
                    ParameterScan\Alpha1'
118             #File_path = 'E:\Multi\ParameterScan\Alpha1'
119
120         FileGeneralInfo_path_ini = os.path.join(File_path, "
                Initial_Info.txt") # Path to the general file "
                Epithelial_Multi.txt".
121         txt_GeneralInfo_ini = open(FileGeneralInfo_path_ini, "a") #
                 Opens the file "Epithelial_Multi.txt" to write in.
122
123         Cells_str = ''
124         Cells_str +=
                "-----------------------------------------------------\n"
125         Cells_str += "The initial properties are: \n"
126         Cells_str += "Temperature = " + str(val_Temperature) + " \n
                " +\
127                      "Lambda_Adh_Normal_Normal = " + str(
                            val_ContactEnergy_NormalNormal) + "\n" +\
128                      "Lambda_Adh_Normal_Membrane = " + str(
                            val_ContactEnergy_MembraneNormal) + "\n" +\
129                      "Lambda_Adh_Normal_Medium = " + str(
                            val_ContactEnergy_MediumNormal) + "\n" +\
130                      "Total number of MCS = " + str(val_Steps) + "\
                            n" +\
131                      "MCS_Start_Proliferation = " + str(
                            MCS_Start_Proliferation) + "\n" +\
132                      "Dimension_y = " + str(val_Dimension_y) + "
                            pixels \n" +\
133                      "n_Hill for P_div = " + str(n_Hill)+ "\n" +\
134                      "threshold_Hill for P_div = " + str(
                            threshold_Hill)+ " * " + str(math.sqrt(
                            targetVol_Normal)) + "\n" +\
135                      "Division orientation is: " + str(Div_Ori) +
                            "\n" +\
136                      "Alpha = " + str(Alpha)+ "\n"
137         txt_GeneralInfo_ini.write(Cells_str) # Writes the content
                of "Cells_str" in the file "Initial_Volume.txt".
138
139         for cell in self.cellList:
140             if cell.type == 1 or cell.type == 2:
```

```
                Cells_str = ''
                Cells_str += "Cell ID " + str(cell.id) + \
                             "; Type=" + str(cell.type) + \
                             "; COM=(" + str(round(cell.xCOM, 3)) + "," + str(round(cell.yCOM, 3)) + "); " +\
                             "Lambda_Vol=" + str(cell.lambdaVolume) + "; " +\
                             "Lambda_Surf=" + str(cell.lambdaSurface) + "; " +\
                             "\n"
                txt_GeneralInfo_ini.write(Cells_str) # Writes the content of "Cells_str" in the file "Epithelial_Multi.txt".
        Cells_str = "-----------------------------------------------------\n\n"
        txt_GeneralInfo_ini.write(Cells_str)
        txt_GeneralInfo_ini.close()

        File_Num_Total_ProfStarts = os.path.join(File_path, "MatlabNumTotalProfStarts.txt") # Path to the general file
        txt_Matlab_Num_ProfStarts = open(File_Num_Total_ProfStarts, "a") # Opens the file
        txt_Matlab_Num_ProfStarts.write(str(val_Steps) + "\n")
        txt_Matlab_Num_ProfStarts.write(str(MCS_Start_Proliferation) + "\n")
        txt_Matlab_Num_ProfStarts.close()
        # $$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
        pass

    #..........................
    #..........................
    #..........................
    def step(self,mcs):
        #**************************XML Info*************************************
        pottsXMLData=self.simulator.getCC3DModuleData("Potts")
        if pottsXMLData:
             Dimension_y=pottsXMLData.getFirstElement("Dimensions", d2mss({"x":"480"}))
             if Dimension_y:
                  val_Dimension_y = Dimension_y.getAttributeAsDouble("y")
        #***************************************************************************

        File_path = 'C:\Users\s4392898\Desktop\Multi'
        #File_path = 'E:\Multi'
        if Alpha == 1:
           File_path = 'C:\Users\s4392898\Desktop\Multi\ParameterScan\Alpha1'

        #........Track pixels of a cell.....
        #..................................
        FilePixlesCell = os.path.join(File_path, "MatlabPixelsCell.txt")
        txt_Matlab_PixlesCell = open(FilePixlesCell, "a")
        Cells_str = ''
        Cells_str += "*********************\n"
        Cells_str += "MCS=" + str(mcs) + "\n"
        txt_Matlab_PixlesCell.write(Cells_str)

        self.cell_field = self.potts.getCellFieldG() #loads the cell-lattice parameters.
```

```
187         for cell in self.cellList:
188             if cell.type == 1:
189                 Cells_str = ''
190                 cell.dict['NPixelsTotal'] = 0
191                 cell.dict['NPixelsAdjToFree'] = 0
192                 cell.dict['NPixelsAdjToSubstrate'] = 0
193                 cell.dict['NPixelsAdjToFreeSubstrate'] = 0
194                 boundaryPixelList = self.getCellBoundaryPixelList(
                        cell)
195                 for boundaryPixelData in boundaryPixelList:
196                     pt = boundaryPixelData.pixel  # pt = (
                            pixel_pos_x, pixel_pos_y, 0)
197                     cell.dict['NPixelsTotal'] = cell.dict['
                            NPixelsTotal'] + 1
198                     (x1,y1,z1)=(boundaryPixelData.pixel.x,
                            boundaryPixelData.pixel.y, boundaryPixelData
                            .pixel.z)
199                     #Cells_str += "pt=" + str((x1,y1,z1)) + "\n"
200                     tag_pix_free = 0
201                     tag_pix_substrate = 0
202                     tag_pix_free_substrate = 0
203                     for (dx,dy,dz) in ((1,0,0),(0,1,0),(-1,0,0)
                            ,(0,-1,0)):
204                         cell_2 = self.cell_field[x1+dx, y1+dy, z1+
                                dz]
205                         if cell_2:
206                             if cell_2.type == 4:
207                                 #cell.dict['NPixelsAdjToSubstrate']
                                        = cell.dict['
                                            NPixelsAdjToSubstrate'] + 1
208                                 #Cells_str += "pt is adjacent to
                                        the Substrate. \n"
209                                 tag_pix_substrate = 1
210                             #pass
211                             #if true, the neighbour (cell_2) is a
                                    cell. If false, cell_2 is medium.
212                         else:
213                             #cell.dict['NPixelsAdjToFree']= cell.
                                    dict['NPixelsAdjToFree'] +1
214                             #Cells_str += "pt is adjacent to the
                                    Medium. \n"
215                             tag_pix_free = 1
216                     if tag_pix_substrate == 1:
217                         cell.dict['NPixelsAdjToSubstrate'] = cell.
                                dict['NPixelsAdjToSubstrate'] + 1
218                     if tag_pix_free == 1:
219                         cell.dict['NPixelsAdjToFree']= cell.dict['
                                NPixelsAdjToFree'] +1
220                     if tag_pix_substrate == 1 and tag_pix_free ==
                            1:
221                         cell.dict['NPixelsAdjToFreeSubstrate']=
                                cell.dict['NPixelsAdjToFreeSubstrate']
                                +1
222                     #Cells_str += "N_B_Pix(tot)=" + str(cell.dict['
                            NPixelsTotal']) +\
223                     #         ", N_B_Pix(free)=" + str(cell.dict['
                            NPixelsAdjToFree']) +\
224                     #         ", N_B_pix(sub)=" + str(cell.dict['
                            NPixelsAdjToSubstrate']) +\
225                     #         ", N_B_pix(sub-free)=" + str(cell.
                            dict['NPixelsAdjToFreeSubstrate']) +\
226                     #         "\n"
227
228                 Cells_str += "Cell.ID=" + str(cell.id)+\
229                              ", xCOM=" + str(round(cell.xCOM,3))+\
230                              ", yCOM=" + str(round(cell.xCOM,3))+\
231                              ", N_B_Pix(tot)=" + str(cell.dict['
```

```
                                    NPixelsTotal']) +\
232                                 ", N_B_Pix(free)=" + str(cell.dict['
                                    NPixelsAdjToFree']) +\
233                                 ", N_B_pix(sub)=" + str(cell.dict['
                                    NPixelsAdjToSubstrate']) +\
234                                 ", N_B_pix(sub-free)=" + str(cell.dict
                                    ['NPixelsAdjToFreeSubstrate']) +\
235                                 "\n"
236                 txt_Matlab_PixlesCell.write(Cells_str)
237         txt_Matlab_PixlesCell.close()
238         #.................................
239         #.................................
240
241         #......... Number, velocity, and ecc of cells at the edge
                and all cells ........
242 #         if mcs == MCS_Start_Proliferation: # Initialising
    velocity.
243 #             for cell in self.cellList:
244 #                 if cell.type == 1:
245 #                     cell.dict['PreviousComX'] = cell.xCOM
246 #                     cell.dict['PreviousComY'] = cell.yCOM
247
248         if (mcs >= 0):
249             File_Num_Cells = os.path.join(File_path, "
                    MatlabNumCellsMCS.txt") # Path to the general file
250             txt_Matlab_Num_Cells = open(File_Num_Cells, "a") #
                    Opens the file
251             FileNCellsAtEdge_path = os.path.join(File_path, "
                    MatlabNCellsAtEdge.txt")
252             txt_Matlab_NCellsAtEdge = open(FileNCellsAtEdge_path, "
                    a")
253             FileNCellsBasal_path = os.path.join(File_path, "
                    MatlabNCellsBasal.txt")
254             txt_Matlab_NCellsBasal = open(FileNCellsBasal_path, "a
                    ")
255             FileNCellsSuprabasal_path = os.path.join(File_path, "
                    MatlabNCellsSuprabasal.txt")
256             txt_Matlab_NCellsSuprabasal = open(
                    FileNCellsSuprabasal_path, "a")
257
258             FileCOM_y_Basal_path = os.path.join(File_path, "
                    COMyBasal.txt")
259             txt_Matlab_COMyBasal = open(FileCOM_y_Basal_path, "a")
260
261             FileNpixelSubTotalAllBasal_path = os.path.join(
                    File_path, "NPixelTotSub_Basal_MCS.txt")
262             txt_Matlab_NpixelSubTotalAllBasal = open(
                    FileNpixelSubTotalAllBasal_path, "a")
263
264             FileyCOMEdge_path = os.path.join(File_path, "
                    MatlabyCOMEdge.txt")
265             txt_Matlab_yCOMEdge = open(FileyCOMEdge_path, "a")
266
267             FileEccAll_path = os.path.join(File_path, "MatlabEccAll
                    .txt")
268             txt_Matlab_EccAll = open(FileEccAll_path, "a")
269             FileEccEdge_path = os.path.join(File_path, "
                    MatlabEccEdge.txt")
270             txt_Matlab_EccEdge = open(FileEccEdge_path, "a")
271             FileEccCellsBasal_path = os.path.join(File_path, "
                    MatlabEccCellsBasal.txt")
272             txt_Matlab_EccCellsBasal = open(FileEccCellsBasal_path,
                     "a")
273             FileEccCellsSuprabasal_path = os.path.join(File_path, "
                    MatlabEccCellsSuprabasal.txt")
274             txt_Matlab_EccCellsSuprabasal = open(
```

```
                FileEccCellsSuprabasal_path, "a")
275
276             FileVolAll_path = os.path.join(File_path, "MatlabVolAll
                    .txt")
277             txt_Matlab_VolAll = open(FileVolAll_path, "a")
278             FileVolEdge_path = os.path.join(File_path, "
                    MatlabVolEdge.txt")
279             txt_Matlab_VolEdge = open(FileVolEdge_path, "a")
280             FileVolBasal_path = os.path.join(File_path, "
                    MatlabVolBasal.txt")
281             txt_Matlab_VolBasal = open(FileVolBasal_path, "a")
282             FileVolSuprabasal_path = os.path.join(File_path, "
                    MatlabVolSuprabasal.txt")
283             txt_Matlab_VolSuprabasal = open(FileVolSuprabasal_path,
                     "a")
284
285             FileVolTissue_path = os.path.join(File_path, "
                    MatlabVolTissue.txt")
286             txt_Matlab_VolTissue = open(FileVolTissue_path, "a")
287
288             FileOrderCOMV_path = os.path.join(File_path, "
                    TrackCOM_V.txt")
289             txt_TrackCOM_V = open(FileOrderCOMV_path, "a")
290
291             FileConfluent_path = os.path.join(File_path, "Confluent
                    .txt")
292             txt_Confluent = open(FileConfluent_path, "a")
293
294 #             FileOrderV_path = os.path.join(File_path, "
    MatlabOrderV.txt")
295 #             txt_Matlab_OV_AllCells = open(FileOrderV_path, "a")
296
297 #             FileVEdge_path = os.path.join(File_path, "MatlabVEdge
    .txt")
298 #             txt_Matlab_VEdge = open(FileVEdge_path, "a")
299
300
301
302             for cell in self.cellList:
303                 cell.dict['IsConnectedToMedium'] = 0
304                 cell.dict['IsConnectedToSheet'] = 0
305                 cell.dict['IsConnectedToMembrane'] = 0
306                 if cell.type == 1:
307                     for neighbor , commonSurfaceArea in self.
                            getCellNeighborDataList(cell):
308                         if neighbor == None:
309                             cell.dict['IsConnectedToMedium'] = 1
310                         if neighbor != None:
311                             cell.dict['IsConnectedToSheet'] = 1
312                             if neighbor.type == 4:
313                                 cell.dict['IsConnectedToMembrane']
                                        = 1
314
315             n_AllCells_MCS = 0
316             n_cells_atEdge_MCS = 0
317             n_cells_Basal_MCS = 0
318             n_cells_Suprabasal = 0
319
320             Sum_velocity_cells_atEdge = 0
321             Average_Velocity_Edge = 0
322
323             ECC_AllCells_MCS = []
324             Sum_Ecc_AllCells = 0
325             Average_Ecc_AllCells = 0
326             std_Ecc_AllCells = 0
327
328             ECC_atEdge_MCS = []
329             Sum_Ecc_cells_atEdge = 0
```

```
330             Average_Ecc_Edge = 0
331             std_Ecc_Edge = 0
332
333             ECC_Basal_MCS = []
334             Sum_Ecc_cells_Basal = 0
335             Average_Ecc_Basal = 0
336             std_Ecc_Basal = 0
337
338             ECC_Suprabasal_MCS = []
339             Sum_Ecc_cells_Suprabasal = 0
340             Average_Ecc_Suprabasal = 0
341             std_Ecc_Suprabasal = 0
342
343             Vol_AllCells_MCS = []
344             Sum_Vol_AllCells = 0
345             Average_Vol_AllCells = 0
346             std_Vol_AllCells = 0
347
348             Vol_atEdge_MCS = []
349             Sum_Vol_cells_atEdge = 0
350             Average_Vol_Edge = 0
351             std_Vol_Edge = 0
352
353             Vol_Basal_MCS = []
354             Sum_Vol_cells_Basal = 0
355             Average_Vol_Basal = 0
356             std_Vol_Basal = 0
357
358             Vol_Suprabasal_MCS = []
359             Sum_Vol_cells_Suprabasal = 0
360             Average_Vol_Suprabasal = 0
361             std_Vol_Suprabasal = 0
362
363             COMy_atEdge_MCS = []
364             Sum_yCOM_cells_atEdge = 0
365             Average_yCOM_Edge = 0
366
367             COMy_Basal_MCS = []
368             Sum_COM_y_BasalCells = 0
369             Average_COM_y_BasalCells = 0
370             std_COM_y_BasalCells = 0
371
372             NPixelsSubst_Basal_MCS = []
373             NPixelsTotal_Basal_MCS = []
374
375             confluent_tag = 1
376
377             Tissue_Vol = 0
378
379             sum_Vmag_AllCells = 0
380             sum_Vx_AllCells = 0
381             sum_Vy_AllCells = 0
382             OrderV_MCS_AllCells = 0
383
384             for cell in self.cellList:
385                 if cell.type == 4:
386                     for neighbor , commonSurfaceArea in self.
                            getCellNeighborDataList(cell):
387                         if neighbor == None:
388                             confluent_tag = 0
389                 if cell.type == 1:
390                     n_AllCells_MCS = n_AllCells_MCS + 1
391
392                     Vol_AllCells_MCS.append(cell.volume)
393                     Sum_Vol_AllCells = Sum_Vol_AllCells + cell.
                            volume
394                     if cell.ecc > 0:
395                         ECC_AllCells_MCS.append(cell.ecc)
396                         Sum_Ecc_AllCells = Sum_Ecc_AllCells + cell.
                                ecc
397                     if cell.ecc == 0:
```

```
398                             axes=self.momentOfInertiaPlugin.getSemiaxes
                                    (cell) # axes is a 3-component vector.
                                    axes[0]=length of minor axis. axes[1]=
                                    length of median axis (which is set to 0
                                     in 2D). axes[2]=the length of major
                                    semiaxis.
399                             temp_ecc = math.sqrt(1 - ((axes[0]**2)/(
                                    axes[2]**2)))
400                             ECC_AllCells_MCS.append(temp_ecc)
401                             Sum_Ecc_AllCells = Sum_Ecc_AllCells +
                                    temp_ecc
402                         Tissue_Vol = Tissue_Vol + cell.volume
403 #                         cell.dict['velocityX'] = cell.xCOM - cell.
    dict['PreviousComX'] #Vx = comX - previousComX. For the order
    parameter.
404 #                         cell.dict['velocityY'] = cell.yCOM - cell.
    dict['PreviousComY'] #Vy = comY - previousComY. For the order
    parameter.
405                         txt_TrackCOM_V.write("MCS=" + str(mcs) +\
406                                              ", ID=" + str(cell.id) +\
407                                              ", cell.yCOM=" + str(cell.
                                                    yCOM) +\
408                                              " \n")
409 #                                            ", cell.dict['velocityY
    ']=" + str(cell.dict['velocityY']) +\
410 #                                              ", cell.dict['PreComY
    ']=" + str(cell.dict['PreviousComY']) +\
411
412
413 #                         cell.dict['PreviousComX'] = cell.xCOM #
    previousComX = Xcom
414 #                         cell.dict['PreviousComY'] = cell.yCOM #
    previousComY = Ycom
415 #                         cell.dict['VelocityMagnitude'] = math.sqrt(
    cell.dict['velocityX']**2 + cell.dict['velocityY']**2) # |V|.
    For the order parameter.
416 #                         sum_Vmag_AllCells = sum_Vmag_AllCells + cell.
    dict['VelocityMagnitude'] # for the order parameter.
417 #                         sum_Vx_AllCells = sum_Vx_AllCells + cell.dict
    ['velocityX'] # for the order parameter.
418 #                         sum_Vy_AllCells = sum_Vy_AllCells + cell.dict
    ['velocityY'] # for the order parameter.
419                         if cell.dict['IsConnectedToSheet'] == 1 and
                                cell.dict['IsConnectedToMedium'] == 1:
420                             n_cells_atEdge_MCS = n_cells_atEdge_MCS + 1
421                             #Sum_velocity_cells_atEdge =
                                    Sum_velocity_cells_atEdge + cell.dict['
                                    velocityY']
422                             #Sum_Ecc_cells_atEdge =
                                    Sum_Ecc_cells_atEdge + cell.ecc
423                             if cell.ecc > 0:
424                                 ECC_atEdge_MCS.append(cell.ecc)
425                                 Sum_Ecc_cells_atEdge =
                                        Sum_Ecc_cells_atEdge + cell.ecc
426                             if cell.ecc == 0:
427                                 axes=self.momentOfInertiaPlugin.
                                        getSemiaxes(cell) # axes is a 3-
                                        component vector. axes[0]=length of
                                        minor axis. axes[1]=length of median
                                         axis (which is set to 0 in 2D).
                                        axes[2]=the length of major semiaxis
                                        .
428                                 temp_ecc = math.sqrt(1 - ((axes[0]**2)
                                        /(axes[2]**2)))
429                                 ECC_atEdge_MCS.append(temp_ecc)
430                                 Sum_Ecc_cells_atEdge =
                                        Sum_Ecc_cells_atEdge + temp_ecc
```

```
431                         Vol_atEdge_MCS.append(cell.volume)
432                         Sum_Vol_cells_atEdge = Sum_Vol_cells_atEdge
                                + cell.volume
433                         COMy_atEdge_MCS.append(cell.yCOM)
434                         Sum_yCOM_cells_atEdge =
                               Sum_yCOM_cells_atEdge + cell.yCOM
435                     if cell.dict['IsConnectedToMembrane'] == 1:
436                         n_cells_Basal_MCS = n_cells_Basal_MCS + 1
437                         #Sum_Ecc_cells_Basal = Sum_Ecc_cells_Basal
                               + cell.ecc
438                         if cell.ecc > 0:
439                             ECC_Basal_MCS.append(cell.ecc)
440                             Sum_Ecc_cells_Basal =
                                   Sum_Ecc_cells_Basal + cell.ecc
441                         if cell.ecc == 0:
442                             axes=self.momentOfInertiaPlugin.
                                   getSemiaxes(cell) # axes is a 3-
                                   component vector. axes[0]=length of
                                   minor axis. axes[1]=length of median
                                    axis (which is set to 0 in 2D).
                                   axes[2]=the length of major semiaxis
                                   .
443                             temp_ecc = math.sqrt(1 - ((axes[0]**2)
                                   /(axes[2]**2)))
444                             ECC_Basal_MCS.append(temp_ecc)
445                             Sum_Ecc_cells_Basal =
                                   Sum_Ecc_cells_Basal + temp_ecc
446                         Vol_Basal_MCS.append(cell.volume)
447                         Sum_Vol_cells_Basal = Sum_Vol_cells_Basal +
                                cell.volume
448                         COMy_Basal_MCS.append(cell.yCOM)
449                         Sum_COM_y_BasalCells = Sum_COM_y_BasalCells
                                + cell.yCOM
450                         NPixelsSubst_Basal_MCS.append(cell.dict['
                               NPixelsAdjToSubstrate'])
451                         NPixelsTotal_Basal_MCS.append(cell.dict['
                               NPixelsTotal'])
452                     if cell.dict['IsConnectedToMembrane'] == 0:
453                         n_cells_Suprabasal = n_cells_Suprabasal + 1
454                         #Sum_Ecc_cells_Suprabasal =
                               Sum_Ecc_cells_Suprabasal + cell.ecc
455                         if cell.ecc > 0:
456                             ECC_Suprabasal_MCS.append(cell.ecc)
457                             Sum_Ecc_cells_Suprabasal =
                                   Sum_Ecc_cells_Suprabasal + cell.ecc
458                         if cell.ecc == 0:
459                             axes=self.momentOfInertiaPlugin.
                                   getSemiaxes(cell) # axes is a 3-
                                   component vector. axes[0]=length of
                                   minor axis. axes[1]=length of median
                                    axis (which is set to 0 in 2D).
                                   axes[2]=the length of major semiaxis
                                   .
460                             temp_ecc = math.sqrt(1 - ((axes[0]**2)
                                   /(axes[2]**2)))
461                             ECC_Suprabasal_MCS.append(temp_ecc)
462                             Sum_Ecc_cells_Suprabasal =
                                   Sum_Ecc_cells_Suprabasal + temp_ecc
463                         Vol_Suprabasal_MCS.append(cell.volume)
464                         Sum_Vol_cells_Suprabasal =
                               Sum_Vol_cells_Suprabasal + cell.volume
465
466                 if n_AllCells_MCS > 0:
467                     #Average_Ecc_AllCells = float(Sum_Ecc_AllCells) /
                           float(n_AllCells_MCS)
468                     #Average_Vol_AllCells = float(Sum_Vol_AllCells) /
                           float(n_AllCells_MCS)
469                     txt_Matlab_EccAll.write(str(mcs) + " " + str(np.
```

```
                mean(ECC_AllCells_MCS)) + " " + str(np.std(
                ECC_AllCells_MCS)) + "\n")
470             txt_Matlab_VolAll.write(str(mcs) + " " + str(np.
                mean(Vol_AllCells_MCS)) + " " + str(np.std(
                Vol_AllCells_MCS)) + "\n")
471         else:
472             txt_Matlab_EccAll.write(str(mcs) + " " + str(0) + "
                 " + str(0) + "\n")
473             txt_Matlab_VolAll.write(str(mcs) + " " + str(0) + "
                 " + str(0) + "\n")
474
475         if n_cells_Basal_MCS > 0:
476             #Average_Ecc_Basal = float(Sum_Ecc_cells_Basal) /
                float(n_cells_Basal_MCS)
477             #Average_Vol_Basal = float(Sum_Vol_cells_Basal) /
                float(n_cells_Basal_MCS)
478             #Average_COM_y_BasalCells = float(
                Sum_COM_y_BasalCells) / float(n_cells_Basal_MCS)
479             txt_Matlab_EccCellsBasal.write(str(mcs) + " " + str
                (np.mean(ECC_Basal_MCS)) + " " + str(np.std(
                ECC_Basal_MCS)) + "\n")
480             txt_Matlab_VolBasal.write(str(mcs) + " " + str(np.
                mean(Vol_Basal_MCS)) + " " + str(np.std(
                Vol_Basal_MCS)) + "\n")
481             txt_Matlab_COMyBasal.write(str(mcs) + " " + str(np.
                mean(COMy_Basal_MCS)) + " " + str(np.std(
                COMy_Basal_MCS)) +"\n")
482             txt_Matlab_NpixelSubTotalAllBasal.write(str(mcs) +
                " " + str(np.mean(NPixelsSubst_Basal_MCS)) + " "
                 + str(np.std(NPixelsSubst_Basal_MCS)) + " " +
                str(np.mean(NPixelsTotal_Basal_MCS)) + " "  +
                str(np.std(NPixelsTotal_Basal_MCS))+ "\n")
483         else:
484             txt_Matlab_EccCellsBasal.write(str(mcs) + " " + str
                (0) + " " + str(0) + "\n")
485             txt_Matlab_VolBasal.write(str(mcs) + " " + str(0) +
                 " " + str(0) + "\n")
486             txt_Matlab_COMyBasal.write(str(mcs) + " " + str(0)
                + " " + str(0) +"\n")
487
488         if n_cells_Suprabasal > 0:
489             #Average_Ecc_Suprabasal = float(
                Sum_Ecc_cells_Suprabasal) / float(
                n_cells_Suprabasal)
490             #Average_Vol_Suprabasal = float(
                Sum_Vol_cells_Suprabasal)/ float(
                n_cells_Suprabasal)
491             txt_Matlab_EccCellsSuprabasal.write(str(mcs) + " "
                + str(np.mean(ECC_Suprabasal_MCS)) + " " + str(
                np.std(ECC_Suprabasal_MCS)) + "\n")
492             txt_Matlab_VolSuprabasal.write(str(mcs) + " " + str
                (np.mean(Vol_Suprabasal_MCS)) + " " + str(np.std
                (Vol_Suprabasal_MCS)) + "\n")
493         else:
494             txt_Matlab_EccCellsSuprabasal.write(str(mcs) + " "
                + str(0) + " " + str(0) + "\n")
495             txt_Matlab_VolSuprabasal.write(str(mcs) + " " + str
                (0) + " " + str(0) + "\n")
496
497         if n_cells_atEdge_MCS > 0:
498             #Average_Ecc_Edge = float(Sum_Ecc_cells_atEdge) /
                float(n_cells_atEdge_MCS)
499             #Average_Vol_Edge = float(Sum_Vol_cells_atEdge) /
                float(n_cells_atEdge_MCS)
500             #Average_yCOM_Edge = float(Sum_yCOM_cells_atEdge) /
```

```
                    float(n_cells_atEdge_MCS)
#                 Average_Velocity_Edge = Sum_velocity_cells_atEdge
    / n_cells_atEdge_MCS
                txt_Matlab_EccEdge.write(str(mcs) + " " + str(np.
                    mean(ECC_atEdge_MCS)) + " " + str(np.std(
                    ECC_atEdge_MCS)) + "\n")
                txt_Matlab_VolEdge.write(str(mcs) + " " + str(np.
                    mean(Vol_atEdge_MCS)) + " " + str(np.std(
                    Vol_atEdge_MCS)) + "\n")
                txt_Matlab_yCOMEdge.write(str(mcs) + " " + str(np.
                    mean(COMy_atEdge_MCS)) + " " + str(np.std(
                    COMy_atEdge_MCS)) +"\n")
            else:
                txt_Matlab_EccEdge.write(str(mcs) + " " + str(0) +
                    " " + str(0) + "\n")
                txt_Matlab_VolEdge.write(str(mcs) + " " + str(0) +
                    " " + str(0) + "\n")
                txt_Matlab_yCOMEdge.write(str(mcs) + " " + str(0) +
                     " " + str(0) +"\n")

            txt_Matlab_Num_Cells.write(str(mcs) + " " + str(
                n_AllCells_MCS) + "\n")
            txt_Matlab_Num_Cells.close()
            txt_Matlab_NCellsAtEdge.write(str(mcs) + " " + str(
                n_cells_atEdge_MCS) + "\n")
            txt_Matlab_NCellsAtEdge.close()
            txt_Matlab_NCellsBasal.write(str(mcs) + " " + str(
                n_cells_Basal_MCS) + "\n")
            txt_Matlab_NCellsBasal.close()
            txt_Matlab_NCellsSuprabasal.write(str(mcs) + " " + str(
                n_cells_Suprabasal) + "\n")
            txt_Matlab_NCellsSuprabasal.close()


            #txt_Matlab_EccAll.write(str(mcs) + " " + str(np.mean(
                ECC_AllCells_MCS)) + " " + str(np.std(
                ECC_AllCells_MCS)) + "\n")
            txt_Matlab_EccAll.close()
            #txt_Matlab_EccEdge.write(str(mcs) + " " + str(np.mean(
                ECC_atEdge_MCS)) + " " + str(np.std(ECC_atEdge_MCS))
                 + "\n")
            txt_Matlab_EccEdge.close()
            #txt_Matlab_EccCellsBasal.write(str(mcs) + " " + str(np
                .mean(ECC_Basal_MCS)) + " " + str(np.std(
                ECC_Basal_MCS)) + "\n")
            txt_Matlab_EccCellsBasal.close()
            #txt_Matlab_EccCellsSuprabasal.write(str(mcs) + " " +
                str(np.mean(ECC_Suprabasal_MCS)) + " " + str(np.std(
                ECC_Suprabasal_MCS)) + "\n")
            txt_Matlab_EccCellsSuprabasal.close()

            #txt_Matlab_VolAll.write(str(mcs) + " " + str(np.mean(
                Vol_AllCells_MCS)) + " " + str(np.std(
                Vol_AllCells_MCS)) + "\n")
            txt_Matlab_VolAll.close()
            #txt_Matlab_VolEdge.write(str(mcs) + " " + str(np.mean(
                Vol_atEdge_MCS)) + " " + str(np.std(Vol_atEdge_MCS))
                 + "\n")
            txt_Matlab_VolEdge.close()
            #txt_Matlab_VolBasal.write(str(mcs) + " " + str(np.mean
                (Vol_Basal_MCS)) + " " + str(np.std(Vol_Basal_MCS))
                + "\n")
            txt_Matlab_VolBasal.close()
            #txt_Matlab_VolSuprabasal.write(str(mcs) + " " + str(np
                .mean(Vol_Suprabasal_MCS)) + " " + str(np.std(
```

```
                Vol_Suprabasal_MCS)) + "\n")
536             txt_Matlab_VolSuprabasal.close()
537
538             #txt_Matlab_yCOMEdge.write(str(mcs) + " " + str(np.mean
                    (COMy_atEdge_MCS)) + " " + str(np.std(
                    COMy_atEdge_MCS)) +"\n")
539             txt_Matlab_yCOMEdge.close()
540
541             #txt_Matlab_COMyBasal.write(str(mcs) + " " + str(np.
                    mean(COMy_Basal_MCS)) + " " + str(np.std(
                    COMy_Basal_MCS)) +"\n")
542             txt_Matlab_COMyBasal.close()
543             txt_Matlab_NpixelSubTotalAllBasal.close()
544
545             txt_Matlab_VolTissue.write(str(mcs) + " " + str(
                    Tissue_Vol) + "\n")
546             txt_Matlab_VolTissue.close()
547
548             txt_Confluent.write(str(mcs) + " " + str(confluent_tag)
                     + "\n")
549             txt_Confluent.close()
550
551             txt_TrackCOM_V.close()
552             #OrderV_MCS_AllCells = math.sqrt(sum_Vx_AllCells**2 +
                    sum_Vy_AllCells**2)/sum_Vmag_AllCells # Order
                    parameter.
553             #txt_Matlab_OV_AllCells.write(str(mcs) + " " + str(
                    round(OrderV_MCS_AllCells, 3)) + "\n")
554             #txt_Matlab_OV_AllCells.close()
555             #txt_Matlab_VEdge.write(str(mcs) + " " + str(
                    Average_Velocity_Edge) + "\n")
556             #txt_Matlab_VEdge.close()
557         #.....................................................................

558         #$$$$$$$$$$$$$$$$$$$$$$$$ Print
                $$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
559         FileAll_MCS = os.path.join(File_path, "All_MCS.txt") # Path
                to the general file
560         txt_All = open(FileAll_MCS, "a") # Opens the file
561
562         Cells_str = ''
563         Cells_str +=
                "-------------------------------------------------\n"
564         Cells_str += "MCS =" + str(mcs)+": \n"
565         txt_All.write(Cells_str)
566         Num_cells_MCS = 0
567         for cell in self.cellList:
568             if cell.type == 1:
569                 Num_cells_MCS = Num_cells_MCS + 1
570                 axes=self.momentOfInertiaPlugin.getSemiaxes(cell) #
                        axes is a 3-component vector. axes[0]=length of
                        minor axis. axes[1]=length of median axis (
                        which is set to 0 in 2D). axes[2]=the length of
                        major semiaxis.
571                 Cells_str = ''
572                 Cells_str += "ID=" + str(cell.id) + \
573                              "; Type=" + str(cell.type) + \
574                              "; Vol=" + str(round(cell.volume,3))
                                    +\
575                              "; L_Vol=" + str(cell.lambdaVolume) +\
576                              "; Surf=" + str(round(cell.surface,3))
                                     +\
577                              "; L_Surf=" + str(cell.lambdaSurface)
                                    +\
578                              "; ecc=" + str(round(cell.ecc,3)) +\
579                              "; ecc_formula=" + str(math.sqrt(1 -
```

```
                                    ((axes[0]**2)/(axes[2]**2)))) +\
580                                 "; axes=" + str(axes) +\
581                                 ", yCOM=" + str(round(cell.yCOM,3)) +\
582                                 ", xCOM=" + str(round(cell.xCOM,3))+\
583                                 ", N_B_Pix(tot)=" + str(cell.dict['
                                    NPixelsTotal']) +\
584                                 ", N_B_Pix(f)=" + str(cell.dict['
                                    NPixelsAdjToFree']) +\
585                                 ", N_B_Pix(s)=" + str(cell.dict['
                                    NPixelsAdjToSubstrate']) +\
586                                 ", Hill_threshold*sqrt(A0)=" + str(
                                    threshold_Hill * math.sqrt(
                                    targetVol_Normal)) +\
587                                 " \n"
588                                 #", N_B_Pix(fs)=" + str(cell.dict['
                                    NPixelsAdjToFreeSubstrate']) +\
589                                 #", N_B_Pix(f/t)=" + str(round(float(
                                    cell.dict['NPixelsAdjToFree'])/
                                    float(cell.dict['NPixelsTotal']),3)
                                    ) +\
590                                 #", N_B_Pix(s/t)=" + str(round(float(
                                    cell.dict['NPixelsAdjToSubstrate'])
                                    /float(cell.dict['NPixelsTotal'])
                                    ,3)) +\
591                                 #", N_B_Pix(fs/t)=" + str(round(float(
                                    cell.dict['
                                    NPixelsAdjToFreeSubstrate'])/float(
                                    cell.dict['NPixelsTotal']),3)) +\
592                                 #", cell.dict['velocityY']=" + str(
                                    cell.dict['velocityY']) +\
593                                 #
594                                 #", P=1-exp(-tot/med)=" + str(round(1-
                                    math.exp((-1)*float(cell.dict['
                                    NPixelsAdjacentToMedium'])/float(
                                    cell.dict['NPixelsTotal'])),3)) +\
595
596
597                 txt_All.write(Cells_str)
598         Cells_str = ''
599         Cells_str += "\n Number of cells = " + str(Num_cells_MCS) +
                ". \n"
600         txt_All.write(Cells_str)
601         txt_All.close()
602         #$$$$$$$$$$$$$$$$$$$$$$$$$ Print
               $$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$
603         pass
604     #.........................
605     #.........................
606     #.........................
607     def finish(self):
608         # Finish Function gets called after the last MCS
609         pass
610
611 #===================Cell Type Classification=====================
612 from PySteppables import *
613 import CompuCell
614 import sys
615 from PlayerPython import *
616 import CompuCellSetup
617 from math import *
618 from PlayerPython import *
619 class CellTypeVisualizationSteppable(SteppableBasePy):
620     def __init__(self,_simulator,_frequency=1):
621         SteppableBasePy.__init__(self,_simulator,_frequency)
622     def setScalarField(self,_field): #I added this function from
           the manual file for Python scripting 3.6 page 21.
623         self.scalarField=_field
```

```
624      def step(self,mcs):
625          clearScalarValueCellLevel(self.scalarField)
626          for cell in self.cellList:
627              if cell.type==1: # Mutated.
628                  fillScalarValueCellLevel(self.scalarField,cell,
                        cell.dict["Mutation"])
629      def finish(self):
630          return
631
632  # #==================Velocity Vector Field====================
633  # from PySteppables import *
634  # import CompuCell
635  # import sys
636  # from PlayerPython import *
637  # import CompuCellSetup
638  # from math import *
639  # from PlayerPython import *
640  # class VectorFieldCellLevelVisualizationSteppable(SteppableBasePy)
       :
641  #     def __init__(self,_simulator,_frequency=1):
642  #         SteppableBasePy.__init__(self,_simulator,_frequency)
643  #     def setVectorField(self,_field): #I added this function from
       the manual file for Python scripting 3.6 page 21.
644  #         self.vectorField=_field
645  #     def step(self,mcs):
646  #         clearVectorCellLevelField(self.vectorField)
647  #         for cell in self.cellList:
648  #             if cell.type==1 or cell.type==2:
649  #                 insertVectorIntoVectorCellLevelField(self.
       vectorField, cell, cell.dict['velocityX'], cell.dict['velocityY
       '], 0.0)
650  #     def finish(self):
651  #         return
652  # #************************Heat map************************
653  # from PySteppables import *
654  # import CompuCell
655  # import sys
656  # from PlayerPython import *
657  # import CompuCellSetup
658  # from math import *
659  # class ScalarFieldVisualizationSteppableP(SteppableBasePy):
660  #     def __init__(self,_simulator,_frequency=1):
661  #         SteppableBasePy.__init__(self,_simulator,_frequency)
662  #     def setScalarField(self,_field):
663  #         self.scalarField=_field
664  #     def step(self,mcs):
665  #         clearScalarValueCellLevel(self.scalarField)
666  #         for cell in self.cellList:
667  #             if cell.type==1:
668  #                 if mcs > MCS_Start_Proliferation:
669  #                     #ratio_1 = float(cell.dict['NPixelsAdjToFree
       ])/float(cell.dict['NPixelsTotal'])
670  #                     #ratio_2 = float(cell.dict['
       NPixelsAdjToSubstrate'])/float(cell.dict['NPixelsTotal'])
671  #                     #ratio_3 = float(cell.dict['
       NPixelsAdjToFreeSubstrate'])/float(cell.dict['NPixelsTotal'])
672  #                     #P_div = (ratio_1 + ratio_2 - ratio_3) * 0.1
673  #                     ratio = float(cell.dict['NPixelsTotal']) *
       float(threshold_Hill);
674  #                     P_nume = float(pow(cell.dict['
       NPixelsAdjToSubstrate'], n_Hill))
675  #                     P_deno = float(pow(cell.dict['
       NPixelsAdjToSubstrate'], n_Hill)) + float(pow(ratio, n_Hill))
676  #                     P_div = (float(P_nume)/float(P_deno)) * 0.1
677  #                 else:
678  #                     P_div = 0
679  #                 fillScalarValueCellLevel(self.scalarField,cell,
       P_div)
```

```
#     def finish(self):
#         return
#.....................................................................

# I need VolumeSteppable class to be able to use the MitosisSteppable class.
class VolumeSteppable(SteppableBasePy):
    def __init__(self,_simulator,_frequency=1):
        SteppableBasePy.__init__(self,_simulator,_frequency)
    def start(self):
        pass

#-------------------------------------
# From: https://compucell3dreferencemanual.readthedocs.io/en/latest/mitosis.html
# class VolumeParamSteppable(SteppableBasePy):
#     def __init__(self, _simulator, _frequency=1):
#         SteppableBasePy.__init__(self, _simulator, _frequency)

#     def start(self):
#         for cell in self.cellList:
#             cell.targetVolume = 25
#             cell.lambdaVolume = 2.0

#     def step(self, mcs):
#         for cell in self.cellList:
#             cell.targetVolume += 1
#-------------------------------------
#.....................................................................

#********************** Cell mitosis (proliferation) **********************
from PySteppables import *
import CompuCell
import sys
import random
from PySteppablesExamples import MitosisSteppableBase
# MitosisSteppable class: divides cell once it reaches a threshold volume and then adjusts the parameters of the
#                         resulting parent and daughter cells.
#                         This steppable also initializes and appends "MitosisData" objects to the original cell's
#                         (self.parentCell) and daughter cell's (self.childCell) attribute lists. This object contains
#                         information about the time and type of cell division as a list attached to each cell.
class MitosisSteppable(MitosisSteppableBase):
    def __init__(self,_simulator,_frequency=1):
        MitosisSteppableBase.__init__(self,_simulator, _frequency)
        # "_frequency=1" and "(sim,_frequency=1)" in both Python files: we call this steppable every MCS.
        # "_frequency=10" and "(sim,_frequency=10)" in both Python files: this seppable checks every 10 MCS, if cell has reached doubling volume of Target.
        self.setParentChildPositionFlag(0)
        # (): parent child position will be randomized between mitosis event.
        # negative integer - the child cell will always appear on the right of the parent cell.
        # positive integer - the child cell will appear always on the left of the parent cell.
        # 0: parent child position will be randomized.
        #self.isDivided = 0  # ?

    def step(self,mcs):
        File_path = 'C:\Users\s4392898\Desktop\Multi\ParameterScan\Alpha1'
        #File_path = 'E:\Multi\ParameterScan\Alpha1'
        FileDivide_MCS = os.path.join(File_path, "Divide_MCS.txt")
```

```
733         txt_Divide = open(FileDivide_MCS, "a") # Opens the file
734
735         FilePdiv_MCS = os.path.join(File_path, "Pdiv_MCS.txt")
736         txt_Pdiv = open(FilePdiv_MCS, "a") # Opens the file
737
738         FileNPixelTotalSubstrate_MCS = os.path.join(File_path, "
                NPixelTotSub_BasalDivide_MCS.txt")
739         txt_NPixelTotalSubstrate = open(
                FileNPixelTotalSubstrate_MCS, "a") # Opens the file
740
741         FileDivCOM_MCS = os.path.join(File_path, "DivCOM_MCS.txt")
742         txt_DivCOM_MCS = open(FileDivCOM_MCS, "a") # Opens the file
743
744         Cells_str = ''
745         #Cells_str +=
                "-----------------------------------------------------\n"
746         #cells_to_divide = [] # To saves cells that are going to be
                 divided.
747         max_COM_x = 0
748         min_COM_x = 10000
749         if mcs > MCS_Start_Proliferation:
750             #self.isDivided = 0 # Only 1 cell divides per MCS.
751             for cell in self.cellList:
752                 if cell.type == 1:
753                     cell.dict['IsConnectedToMembrane'] = 0
754                     for neighbor , commonSurfaceArea in self.
                            getCellNeighborDataList(cell):
755                         if neighbor != None:
756                             if neighbor.type == 4:
757                                 cell.dict['IsConnectedToMembrane']
                                        = 1
758
759             for cell in self.cellList:
760                 if cell.type == 1 and cell.dict['
                        IsConnectedToMembrane'] == 1:
761                     if cell.xCOM >= max_COM_x:
762                         max_COM_x = cell.xCOM # location of the
                                right boundary cell attached to membrane
                                .
763                     if cell.xCOM <= min_COM_x:
764                         min_COM_x = cell.xCOM # location of the
                                left boundary cell attached to membrane.
765             DivCOM_MCS_str = ''
766             DivCOM_MCS_str += str(mcs) + " " + str(round(min_COM_x
                    ,4)) + " " + str(round(max_COM_x,4)) + " "
767
768             for cell in self.cellList:
769                 if cell.type == 1 and cell.dict['
                        IsConnectedToMembrane'] == 1:
770                     if cell.volume >= (targetVol_Normal *
                            threshold_area_div):
771                         #ratio_1 = float(cell.dict['
                                NPixelsAdjToFree'])/float(cell.dict['
                                NPixelsTotal'])
772                         #ratio_2 = float(cell.dict['
                                NPixelsAdjToSubstrate'])/float(cell.dict
                                ['NPixelsTotal'])
773                         #ratio_3 = float(cell.dict['
                                NPixelsAdjToFreeSubstrate'])/float(cell.
                                dict['NPixelsTotal'])
774                         #P_div = (ratio_1 + ratio_2 - ratio_3) *
                                0.1
775                         #n_Hill = 20
776                         #ratio = float(cell.dict['NPixelsTotal']) *
                                 float(threshold_Hill);
777                         ratio = threshold_Hill * math.sqrt(
                                targetVol_Normal)
778                         P_nume = float(pow(cell.dict['
```

```
                    NPixelsAdjToSubstrate'], n_Hill))
779                 P_deno = float(pow(cell.dict['
                    NPixelsAdjToSubstrate'], n_Hill)) +
                    float(pow(ratio, n_Hill))
780                 P_div = (float(P_nume)/float(P_deno)) * 0.1
781                 rand_divide = random.random() # Return the
                    next random floating point number in the
                     range [0.0, 1.0).
782                 if rand_divide < P_div:
783                     Cells_str = ''
784                     Cells_str += "At MCS=" + str(mcs) +\
785                             ", ID=" + str(cell.id) +\
786                             ": rand_div=" + str(round(
                                rand_divide, 3)) +\
787                             ", P_div=" + str(round(P_div
                                ,3)) + " = [" + str(round(
                                P_nume,3)) + " / (" + str(
                                round(P_nume,3)) + " + " +
                                str(round(float(pow(ratio,
                                n_Hill)),3)) +")]"+\
788                             ", Vol=" + str(round(cell.
                                volume,3))
789                     txt_Pdiv.write(str(mcs) + " " + str(
                        P_div) + "\n")
790                     txt_NPixelTotalSubstrate.write(str(mcs)
                         + " " + str(cell.dict['NPixelsTotal
                        ']) + " " + str(cell.dict['
                        NPixelsAdjToSubstrate']) + " " + str
                        (ratio) + "\n")
791                     DivCOM_MCS_str += str(round(cell.xCOM
                        ,3)) + " "
792                     self.setParentChildPositionFlag(0)
793                     #self.divideCellRandomOrientation(cell)
794                     #self.divideCellAlongMajorAxis(cell)
795                     #self.divideCellAlongMinorAxis(cell)
796                     self.divideCellOrientationVectorBased(
                        cell,0,1,0)
797                     # chooses orientation of division axis.
                         Divides cell into two daughter
                        cells along a plane specified by a
                        normal vector (nx, ny, nz).
798                     # For tracking reasons one daughter
                        cell is considered a "parent" and
                        refers to a cell object that existed
                         before division.
799                     #parameter cell: {CompuCell.CellG} cell
                         to divide.
800                     #parameter nx: {float} 'x' component of
                         vector normal to the plane.
801                     #parameter ny: {float} 'y' component of
                         vector normal to the plane.
802                     #parameter nz: {float} 'z' component of
                         vector normal to the plane.
803                     #self.deleteCell(cell) # delete the
                        dividing cell.
804                     #self.isDivided = 1    # Only 1
                        division occurs per MCS.
805                     #cells_to_divide.append(cell)
806                     #Cells_str = ''
807                     #Cells_str += "At MCS=" + str(mcs) +\
808                     #         ", ID=" + str(cell.id) +\
809                     #         ": rand_div=" + str(round(
                        rand_divide, 3)) +\
810                     #         ", P_div=" + str(round(P_div
                        ,3)) + " = [" + str(round(P_nume,3))
                         + " / (" + str(round(P_nume,3)) + "
```

```
                                    + " + str(round(float(pow(ratio,
                                    n_Hill)),3)) +")]"+\
811                                 Cells_str +=", Vol(parent)=" + str(
                                    round(cell.volume,3))+\
812                                         ", ecc=" + str(round(cell.ecc
                                            ,4)) +\
813                                         ", yCOM=" + str(round(cell.
                                            yCOM,3)) +\
814                                         ", xCOM=" + str(round(cell.
                                            xCOM,3))+\
815                                         " was divided. \n"
816                                 txt_Divide.write(Cells_str)
817             DivCOM_MCS_str += "\n"
818             txt_DivCOM_MCS.write(DivCOM_MCS_str)
819         #Cells_str =
                "-------------------------------------------------\n\n
                "
820         #txt_Divide.write(Cells_str)
821         txt_Divide.close()
822         txt_Pdiv.close()
823         txt_NPixelTotalSubstrate.close()
824         txt_DivCOM_MCS.close()
825
826
827     def updateAttributes(self):
828         #self.cloneParent2Child() # This function is a convenience
                function that copies all parent cell's attributes to
                child cell. "Manual V.3.7.5" page 50.
829         # we manually set parent and child properties like this ["
                Manual V.3.7.5" page 50]:
830         parentCell = self.mitosisSteppable.parentCell
831         childCell = self.mitosisSteppable.childCell
832
833         childCell.type = parentCell.type
834
835         parentCell.targetVolume = targetVol_Normal # 100
836         parentCell.targetSurface = targetSurf_Normal # 0
837
838         childCell.targetVolume = parentCell.targetVolume
839         childCell.lambdaVolume = parentCell.lambdaVolume
840         childCell.targetSurface = parentCell.targetSurface
841         childCell.lambdaSurface = parentCell.lambdaSurface
842
843 #         childCell.dict['velocityX'] = 0
844 #         childCell.dict['velocityY'] = 0
845 #         childCell.dict['VelocityMagnitude'] = 0
846 #         childCell.dict['VelocityAngle'] = 0
847 #         childCell.dict['PreviousComX'] = parentCell.xCOM
848 #         childCell.dict['PreviousComY'] = parentCell.yCOM
849
850         childCell.dict['IsConnectedToSheet'] = 0
851         childCell.dict['IsConnectedToMedium'] = 0
852         childCell.dict['IsConnectedToMembrane'] = 0
853         childCell.dict['NPixelsTotal'] = 1 # to avoid division by
                zero in the field visualiser.
854         childCell.dict['NPixelsAdjToFree'] = 0
855         childCell.dict['NPixelsAdjToSubstrate'] = 0
856         childCell.dict['NPixelsAdjToFreeSubstrate'] = 0
857
858
859 #         if parentCell.type == 1: # Mutated
860 #             childCell.type = 1 # Normal
861 #         else:
862 #             childCell.type = 1
863         #The cell type of one of the two daughter cells (childCell)
              is randomly chosen to be Normal.
864         #if random() < 0.5:
865             #childCell.type = parentCell.type
```

```
866         #else:
867             #childCell.type = self.Mutated
868         File_path = 'C:\Users\s4392898\Desktop\Multi\ParameterScan\
               Alpha1'
869         #File_path = 'E:\Multi\ParameterScan\Alpha1'
870         FileBornCells_MCS = os.path.join(File_path, "BornCells_MCS.
               txt")
871         txt_Born = open(FileBornCells_MCS, "a") # Opens the file
872         Cells_str = ''
873         #Cells_str +=
               "--------------------------------------------------\n"
874         mcs = self.simulator.getStep()
875         Cells_str += "At MCS=" + str(mcs) +\
876                      ": parentCell.ID=" + str(parentCell.id) +\
877                      ", Type=" + str(parentCell.type) +\
878                      ", vol=" + str(parentCell.volume) +\
879                      ", L_Vol=" + str(parentCell.lambdaVolume) +\
880                      ", V_t=" + str(parentCell.targetVolume) +\
881                      "; Surf=" + str(parentCell.surface) +\
882                      ", L_Surf=" + str(parentCell.lambdaSurface) +\
883                      ", Surf_t=" + str(parentCell.targetSurface) +\
884                      "\n" +\
885                      "          childCell.ID = " + str(childCell.
                            id) +\
886                      ", Type=" + str(childCell.type) +\
887                      ", vol=" + str(childCell.volume) +\
888                      ", L_Vol=" + str(childCell.lambdaVolume) +\
889                      ", V_t=" + str(childCell.targetVolume) +\
890                      "; Surf=" + str(childCell.surface) +\
891                      ", L_Surf=" + str(childCell.lambdaSurface) +\
892                      ", Surf_t=" + str(childCell.targetSurface) +\
893                      ". \n"
894                  #                     ", parentCell.dict['
                         PreviousComY']=" + str(parentCell.dict['
                         PreviousComY']) +\
895                  #                     ", childCell.dict['
                         PreviousComY']=" + str(childCell.dict['
                         PreviousComY']) +\
896
897         txt_Born.write(Cells_str)
898         txt_Born.close()
899     def finish(self):
900         return
```

D CC3D Code: Parameter Scan (XML)

```
<!-- ################################################
% # Analysis of the collective self-organisation of epithelial layers.
% # Author: Hamid Khataee.
% # Email: h.khataee@uq.edu.au
% # PLEASE DO NOT REDISTRIBUTE WITHOUT PERMISSION.
% ################################################ -->
<CompuCell3D Revision="20180621" Version="3.7.8">

   <Potts>
      <LatticeType>Hexagonal</LatticeType> <!-- To enable hexagonal lattice. With hexagonal lattice, pixels are hexagons.-->
<!--       <Dimensions x="1500" y="170" z="1"/>  -->
<!--      <Dimensions x="520" y="130" z="1"/>    -->
      <Dimensions x="480" y="195" z="1"/>
<!--       <Dimensions x="1500" y="620" z="1"/>  -->
      <Steps>100000</Steps>
      <Temperature>50</Temperature>
      <NeighborOrder>2</NeighborOrder>
      <Boundary_y>no-flux</Boundary_y>
      <Boundary_x>no-flux</Boundary_x>
      <EnergyFunctionCalculator Type="Statistics">
      <OutputCoreFileNameSpinFlips
      Frequency="100"
      GatherResults=""
      OutputAccepted=""
      OutputRejected=""
      OutputTotal="">
      statDataSingleFlip
      </OutputCoreFileNameSpinFlips>
      </EnergyFunctionCalculator>
   </Potts>

   <!--
   The Freeze=       attribute excludes cells of type Wall from participating in index copies, which makes the wall cells immobile.
   -->
   <Plugin Name="CellType">
      <!-- Listing all cell types in the simulation -->
      <CellType TypeId="0" TypeName="Medium"/>
      <CellType TypeId="1" TypeName="Normal"/>
      <CellType TypeId="2" TypeName="Mutated"/>
      <CellType TypeId="3" TypeName="Wall" Freeze=""/>
      <CellType TypeId="4" TypeName="Membrane"/>
   </Plugin>

   <Plugin Name="BoundaryPixelTracker">
      <NeighborOrder>1</NeighborOrder>
   </Plugin>


   <!--
  In the old version, the first line is: <Plugin Name="VolumeFlex">.
  Initially cells are always square (even with "UniformInitializer").
  Since I have declared hexagonal lattice type, pixels of the cells are always hexagonal.
  So, initial area of the 5*5 cells is equal to 25.
  So, I assign Vt = 25: at time zero, the area ealsticity has no effect on the energy (i.e. no effect on starting the motility).
   -->
```

```
56     <Plugin Name="Volume"/>
57    <!-- I need the line above to be able to set values for
          LambdaVolume and TargetVolume in the Python code.
58   -->
59
60     <!--
61     LambdaSurface:
62     thershold is between 0.1 and 1. For (LambdaSurface > threshold)
          cells shape (hexagons) will not change (very stiff shapes).
63                                      For (LambdaSurface < threshold)
                                           cells shape (hexagons) will
                                           change (soft shapes).
64     For (LambdaSurface >> threshold): the hexagonal cells will not
          change (very stiff) and active motility force will not have
          effect.
65     So, we put LambdaSurface close to threshold to see the dynamics
          due to polarity force.
66     -->
67     <!--
68     <Plugin Name="Surface">
69        <SurfaceEnergyParameters CellType="Normal" TargetSurface="0"
             LambdaSurface="1"/>
70        <SurfaceEnergyParameters CellType="Mutated" TargetSurface="0"
              LambdaSurface="1"/>
71       <SurfaceEnergyParameters CellType="Membrane" TargetSurface="0"
             LambdaSurface="7"/>
72     </Plugin>
73      -->
74     <Plugin Name="Surface"/>
75       <!-- I need the line above to be able to set values for
            LambdaSurface and TargetSurface in the Python code.
76   -->
77
78     <!--
79     To have the contribution of "elactity" in the energy calculation
          : I must specify which cell types participate in the
          elasticity = plasticity calculations.
80     This is done by including "ElasticityTracker" plugin before (
          must be before) "Elasticity" plugin in the CC3DML file.
81     Here cells of type Polarized and Unpolarized will be taken into
          account for elasticity energy calculation purposes (p. 37,
          part 2).
82     Plugin "ElasticityEnergy": puts constraints for the distance
          between cells center of masses.
83     Local: defines lamda and Lt on per pair of cells basis (p. 39
          part 2).
84     Lt = "TargetLengthElasticity": Target distance between center of
           masses of cells neighboring cells i and j.
85     -->
86     <!-- <Plugin Name="ElasticityTracker">
87        <IncludeType>Polarized</IncludeType>
88        <IncludeType>Unpolarized</IncludeType>
89      </Plugin>
90     <Plugin Name="ElasticityEnergy">
91        <Local/>
92        <LambdaElasticity>200.0</LambdaElasticity>
93        <TargetLengthElasticity>15</TargetLengthElasticity>
94     </Plugin> -->
95
96     <!--
97     Similar to "Elasticity" plugin: "FocalPointPlasticity" puts
          constraints for the distance between cells center of
          masses (p. 40, part 2).
98     The main difference is: the list of focal point plasticity
          neighbors can change as the simulation goes and user
          specifies the maximum number of
99                               focal point plasticity neighbors a
                                   given cell can have.
100    "MaxDistance" = the distance between cells COMs when the link
```

```
        between those cells breaks. When the link breaks, in order
        for the two cells to reconnect,
101     they would need to come in contact again (in order to reconnect)
        . However, it is usually more likely that there will be other
        cells in the vicinity of
102     separated cells. So, it is more likely to establish new link
        than restoring the broken one.
103     "ActivationEnergy": is added to overall energy in order to
        increase the odds of pixel copy which would lead to new
        connection.
104     "MaxNumberOfJunctions": max number of connections that a cell
        can have (e.g. Polarised cells can have up to 4 connections
        with Unpolarised cells, and
105
106     "NeighborOrder" = 2nd nearest pixel neighbors will be visited
        for pixel-copy.
107     -->
108     <!--
109     <Plugin Name="FocalPointPlasticity">
110        <Local/>
111        <Parameters Type1="Polarized" Type2="Polarized">
112        <Lambda>10.0</Lambda>
113        <TargetDistance>6</TargetDistance>
114        <MaxDistance>8</MaxDistance>
115        <ActivationEnergy>-1000</ActivationEnergy>
116        <MaxNumberOfJunctions>6</MaxNumberOfJunctions>
117        </Parameters>
118        <Parameters Type1="Polarized" Type2="Unpolarized">
119        <Lambda>10.0</Lambda>
120        <ActivationEnergy>-1000</ActivationEnergy>
121        <TargetDistance>6</TargetDistance>
122        <MaxDistance>8</MaxDistance>
123        <MaxNumberOfJunctions>6</MaxNumberOfJunctions>
124        </Parameters>
125        <Parameters Type1="Unpolarized" Type2="Unpolarized">
126        <Lambda>10.0</Lambda>
127        <ActivationEnergy>-1000</ActivationEnergy>
128        <TargetDistance>6</TargetDistance>
129        <MaxDistance>8</MaxDistance>
130        <MaxNumberOfJunctions>6</MaxNumberOfJunctions>
131        </Parameters>
132     <NeighborOrder>2</NeighborOrder>
133     </Plugin>
134     -->
135
136     <Plugin Name="CenterOfMass">
137        <!-- Module tracking center of mass of each cell -->
138     </Plugin>
139
140     <!--
141     Low contact energy = P cells are adhisive = like to stay
        attached due to low energy.
142     High Contact energy = M and P cells are highly repulsive =
        adhere weakly.
```

```
143
144     Two seperate cells:
145     With any Contact energy, the polarised cell moves only.
146
147     Two joint cells:
148     With (cell-cell contact energy >= 30) and (cell-medium contact
           energy = 0): the polarised cell moves only.
149     With (cell-cell contact energy < 30) and (cell-medium contact
           energy = 0): both cells move.
150
151     The default contact energy between any cell type and the edge of
            the cell lattice is zero [Book-SysBiol].
152     Wall:  we define a surrounding wall to prevent cells from
           sticking to the cell-lattice boundary.
153
154     -->
155     <Plugin Name="Contact">
156        <Energy Type1="Normal" Type2="Normal">-15</Energy>
157        <Energy Type1="Membrane" Type2="Normal">-300</Energy>
158        <Energy Type1="Mutated" Type2="Mutated">0</Energy>
159        <Energy Type1="Mutated" Type2="Normal">0</Energy>
160        <Energy Type1="Membrane" Type2="Mutated">0</Energy>
161        <Energy Type1="Membrane" Type2="Membrane">0</Energy>
162        <Energy Type1="Membrane" Type2="Wall">0</Energy>
163        <Energy Type1="Membrane" Type2="Medium">0</Energy>
164        <Energy Type1="Wall" Type2="Wall">0</Energy>
165        <Energy Type1="Wall" Type2="Medium">0</Energy>
166        <Energy Type1="Wall" Type2="Mutated">0</Energy>
167        <Energy Type1="Wall" Type2="Normal">0</Energy>
168        <Energy Type1="Medium" Type2="Medium">0</Energy>
169        <Energy Type1="Medium" Type2="Mutated">0</Energy>
170        <Energy Type1="Medium" Type2="Normal">0</Energy>
171        <NeighborOrder>2</NeighborOrder>
172     </Plugin>
173
174  <!-- <Plugin Name="ContactLocalProduct"> -->
175  <!-- <Local/> -->
176  <!-- <ContactSpecificity Type1="Medium" Type2="Medium">0</
        ContactSpecificity> -->
177  <!-- <ContactSpecificity Type1="Medium" Type2="Wall">0</
        ContactSpecificity> -->
178  <!-- <ContactSpecificity Type1="Medium" Type2="Barrier">0</
        ContactSpecificity> -->
179  <!-- <ContactSpecificity Type1="Medium" Type2="Polarized">0</
        ContactSpecificity> -->
180  <!-- <ContactSpecificity Type1="Medium" Type2="Unpolarized">0</
        ContactSpecificity> -->
181  <!-- <ContactSpecificity Type1="Polarized" Type2="Polarized">-18</
        ContactSpecificity> -->
182  <!-- <ContactSpecificity Type1="Polarized" Type2="Unpolarized
        ">-18</ContactSpecificity> -->
183  <!-- <ContactSpecificity Type1="Unpolarized" Type2="Unpolarized
        ">-18</ContactSpecificity> -->
184  <!-- <ContactSpecificity Type1="Wall" Type2="Wall">0</
        ContactSpecificity> -->
185  <!-- <ContactSpecificity Type1="Wall" Type2="Barrier">0</
        ContactSpecificity> -->
186  <!-- <ContactSpecificity Type1="Wall" Type2="Polarized">0</
        ContactSpecificity> -->
187  <!-- <ContactSpecificity Type1="Wall" Type2="Unpolarized">0</
        ContactSpecificity> -->
188  <!-- <ContactSpecificity Type1="Barrier" Type2="Barrier">0</
        ContactSpecificity> -->
189  <!-- <ContactSpecificity Type1="Barrier" Type2="Polarized">0</
        ContactSpecificity> -->
190  <!-- <ContactSpecificity Type1="Barrier" Type2="Unpolarized">0</
        ContactSpecificity> -->
```

```
<!-- <ContactFunctionType>Linear</ContactFunctionType> -->
<!-- <EnergyOffset>0.0</EnergyOffset> -->
<!-- <NeighborOrder>2</NeighborOrder> -->
<!-- </Plugin> -->

<!-- FocalPointPlasticity plugin represents adhesion junctions by
   mechanically connecting the centers-of-mass of cells using a
   breakable linear spring.
     Here, EC-EC links (Lambda = 400) are stronger than EC-NV links
         (Lambda = 50) which are, in turn, stronger than NV-NV
        links (Lambda = 20). -->
<!-- FocalPointPlasticity pust constrains the distance between
    cells  center of masses. The list of  focal  point plasticity
     neighbors  can change as the simulation goes, and
     user specifies the maximum number of  focal  point plasticity
         neighbors  a given cell can have. -->
<!-- Parameters describes properties of links between cells (are
   responsible for establishing connections between cells):
Lambda: strength of connection between cells.
ActivationEnergy: is added to overall energy in order to increase
   the odds of pixel copy which would lead to new connection.
TargetDistance: target distance between cell centers.
MaxDistance: determines the distance between cells  center of
   masses when the link between those cells break.
MaxNumberOfJunctions: cell types can have up to
   MaxNumberOfJunctions connections.
NeighborOrder: determines the pixel vicinity of the pixel that is
   about to be overwritten which CC3D will scan in search of the
   new link between cells.
 -->

<!--    <Plugin Name="FocalPointPlasticity"> -->
<!--       <Local/> -->
<!--       <Parameters Type1="Polarized" Type2="Polarized"> -->
<!--       <Lambda>50</Lambda> -->
<!--       <ActivationEnergy>-1000</ActivationEnergy> -->
<!--       <TargetDistance>10</TargetDistance> -->
<!--       <MaxDistance>20</MaxDistance> -->
<!--       <MaxNumberOfJunctions>6</MaxNumberOfJunctions> -->
<!--       </Parameters> -->

<!--       <Parameters Type1="Polarized" Type2="Unpolarized"> -->
<!--       <Lambda>50</Lambda> -->
<!--       <ActivationEnergy>-1000</ActivationEnergy> -->
<!--       <TargetDistance>10</TargetDistance> -->
<!--       <MaxDistance>20</MaxDistance> -->
<!--       <MaxNumberOfJunctions>6</MaxNumberOfJunctions> -->
<!--       </Parameters> -->

<!--       <Parameters Type1="Unpolarized" Type2="Unpolarized"> -->
<!--       <Lambda>50</Lambda> -->
<!--       <ActivationEnergy>-1000</ActivationEnergy> -->
<!--       <TargetDistance>10</TargetDistance> -->
<!--       <MaxDistance>20</MaxDistance> -->
<!--       <MaxNumberOfJunctions>6</MaxNumberOfJunctions> -->
<!--       </Parameters> -->
<!--    <NeighborOrder>1</NeighborOrder> -->
<!--    </Plugin> -->

   <!-- ConnectivityGlobal plugin slows down the simulation. -->
   <Plugin Name="ConnectivityGlobal">
      <Penalty Type="Mutated">1000000000</Penalty>
      <Penalty Type="Normal">1000000000</Penalty>
   </Plugin>

   <!--In deleting a cell: I have to add PixelTracker Plugin to
      CC3DML.-->
   <Plugin Name="PixelTracker"/>
```

```
245     <!--In finding cells at the edge: I have to add NeighborTracker
           to CC3DML.-->
246     <Plugin Name="NeighborTracker"/>
247
248     <!--For calculating eccentricity of cells: I have to add
           MomentOfInertia to CC3DML.-->
249     <Plugin Name="MomentOfInertia"/>
250
251     <Plugin Name="ExternalPotential">
252        <!--
253        This plugin is imposes directed pressure (or force) on cells.
254        To apply external potential (i.e. force) to individual cells,
              in the CC3DML section we specify the pluging and leave it
              empty (i.e. do not assign value to lamda).
255        Then, in the Steppable.py file we change lambdaVecX,
              lambdaVecY, lambdaVecZ, which are properties of cells.
256        Positive component of Lambda vector pushes cell in the
              negative direction and negative component pushes cell in
              the positive direction.(p. 19-20 Part2).
257        -->
258         <!--
259         <ExternalPotentialParameters CellType="Type1" x="-10" y="0"
              z="0"/>
260         <ExternalPotentialParameters CellType="Type2" x="0" y="0" z
              ="0"/>
261         -->
262         <!--  -->
263        <!-- External force applied to cell. Each cell has different
              force and force components have to be managed in Python.
              -->
264        <!-- e.g. cell.lambdaVecX=0.5; cell.lambdaVecY=0.1 ; cell.
              lambdaVecZ=0.3; -->
265        <Algorithm>PixelBased</Algorithm>
266        <!--
267        Calculations done by ExternalPotential Plugin are by default
              based on direction of pixel copy (similarly as in
              chemotaxis plugin). One can however force CC3D to do
268        calculations based on movement of center of mass of cell. We
              use "<Algorithm>PixelBased</Algorithm>" to do calculations
              based on center of mass movement.
269        Remark:Note that in the pixel-based algorithm the typical
              value of pixel displacement used in calculations is of the
              order of 1 (pixel). But, typical displacement of center
              of
270                mass of cell due to single pixel copy is of the order
                      of 1/cell volume (pixels) ~ 0.1 pixel. This implies
                      that to achieve compatible behavior of cells when
                      using center of
271                mass algorithm we need to multiply lambdas by
                      appropriate factor, typically of the order of 10 (p
                      . 20, part 2).
272        -->
273     </Plugin>
274
275
276
277     <!--
278     <Steppable Type="PIFDumper" Frequency= "200">
279        <PIFName>line</PIFName>
280     </Steppable>
281     I commented out this. Steppable "PIFDumper" produces .pif files,
           which cannot be opened.
282           So, I will try to track the COM of cells using a code and
                 save it as a file for every MCS.
283      -->
284
285     <!--
286     <Plugin Name="PlayerSettings">
287       <Project2D XZProj="10"/>
288       <VisualControl ScreenshotFrequency="200" ScreenUpdateFrequency
```

```
          ="10" NoOutput="false" ClosePlayerAfterSimulationDone="
          false" />
289       <Border BorderColor="white" BorderOn="false"/>
290       <Cell Type="1" Color="green"/>
291       <Cell Type="2" Color="red"/>
292     </Plugin>
293     -->
294
295
296 <!--    <Plugin Name="PlayerSettings"> -->
297 <!--      <Project2D XZProj="10"/>  --> -->
298 <!--      <VisualControl outputFrequency="200"/> -->
299 <!--      <VisualControl outputFrequency="200"
        ScreenUpdateFrequency="10" NoOutput="false"
        ClosePlayerAfterSimulationDone="false" />  --> -->
300 <!--      <Border BorderColor="white" BorderOn="false"/>  -->
301 <!--      <Cell Type="1" Color="green"/> -->
302 <!--      <Cell Type="2" Color="red"/>  -->
303 <!--      <Cell Type="3" Color="white"/> -->
304 <!--      <Cell Type="4" Color="blue"/>  -->
305 <!--    </Plugin>    -->
306
307     <Steppable Type="PIFInitializer">
308        <PIFName>Simulation/Epithelial1.piff</PIFName>
309     </Steppable>
310
311
312     <!--
313     "UniformInitializer" tag: to specify rectangular regions of
           field with square (or cube in 3D) cells of user defined types
            (or random types).
314     Box size = X * Y. This means that the box is X*Y pixels. Example
           : page 6 CellDraw manual.
315     "Gap" tag: creates gap space between neighboring cells.
316     "Width" tag: specifies the size of the initial square (cubical
           in 3D) generalized cells.
317                "Width = 5": box is filled with 5x5 (in pixels) cells (
                      initially square) touching each other (Gap=0).
318     "Types" tag: lists the cell types to fill the disk. The types
           have to be separated with ',' and there should be no spaces.
319                   If one of the type names is repeated inside <Types>
                       element, this type will get greater weighting.
                      This means that
320                   the probability of assigning this type to a cell
                      will be greater.
321                   For example <Types>psm,ncad,ncam,ncam,ncam</Types>
                      ncam will assigned to a cell with probability
                      3/5 and psm and ncad with probability 1/5.
322                   When user specifies more than one cell type between
                       <Types> tags then cells for this region will be
                       initialized with types chosen
323                   randomly from the provided list; example: p. 49 -
                      Ref manual.
324     -->
325     <!--
326     <Steppable Type="UniformInitializer">
327     <Region>
328        <BoxMin x="0" y="0" z="0"/>
329        <BoxMax x="50" y="50" z="1"/>
330        <Gap>0</Gap>
331        <Width>5</Width>
332        <Types>Polarized</Types>
333     </Region>
334     </Steppable>
335     -->
336 </CompuCell3D>
```